

Toulouse INP - ENSEEIHT
École Nationale Supérieure d'Électrotechnique, d'Électronique, d'Informatique,
d'Hydraulique et des Télécommunications

RAPPORT DE STAGE DE FIN D'ÉTUDES
effectué chez Actimage

16 mars 2026 - 18 septembre 2026, 6 mois

Ingénierie Logicielle Full-Stack et DevOps

Vianney HERVY - Ingénieur Logiciel

vianney.hervy@etu.toulouse-inp.fr

Toulouse INP - ENSEEIHT
2 rue Charles Camichel
31071 Toulouse

Actimage
5 avenue Franco-Russe
75007 Paris

Responsable académique
Marc PANTEL
Maître de conférence en Informatique à l'IRIT /
ENSEEIHT

Encadrant chez Actimage
David KOLIN
(ex-)Directeur de l'agence Paris-Arcueil

Remerciements

Avant tout, je tiens à remercier mes deux encadrants sur cette expérience enrichissante qu'a été ce stage de fin d'étude. David pour son accompagnement, ses conseils et son humour, Matthias pour sa patience, sa bienveillance et ses succulents repas de fin de semaine.

Mes remerciements s'étendent à l'ensemble de mes collègues de l'agence Actimage Paris-Arcueil et d'ailleurs. Par ordre alphabétique : Aurélie, ma co-stagiaire solidaire pour son amitié, sa politesse et sa conversation ; Marine pour sa jovialité, sa créativité et son attention aux détails ; Romain pour sa curiosité et nos discussions technophiles ; et enfin Thomas, pour sa spontanéité et sa précieuse expertise footballistique.

Mes remerciements vont bien sûr également à Madeleine, ma fiancée, qui y est sûrement pour quelque chose dans la réussite de ce stage.

Sommaire

I	Introduction	6
I.1	Présentation de l'entreprise	6
I.1.1	Composition de l'entreprise	7
I.1.2	Équipe rapprochée	8
I.2	Contexte du stage	9
I.3	Cycle de vie d'un projet	9
I.3.1	Avant-vente et commercialisation	9
I.3.2	Pilotage Itératif et Outillage de Suivi	10
I.3.3	Développement et Assurance Qualité	11
I.3.4	Déploiement, Recette et Maintenance	12
I.3.5	Réalisations	13
I.4	Poste de travail et outillage de développement	14
I.4.1	Environnement d'exécution et philosophie de travail	14
I.4.2	Outils collaboratifs et centralisation de la connaissance	14
I.4.3	Inspection web et débogage dynamique	15
I.4.4	L'éditeur de code: le choix de Neovim	16
I.4.5	Contributions aux logiciels libres et outillage sur mesure	16
I.4.6	Le terminal comme espace de travail unifié	17
I.4.7	Conteneurisation et exécution locale	17
I.4.8	Évolution du poste de travail	17
II	Projet ONaCVG	18
II.1	Introduction	18
II.1.1	Présentation du client et genèse du besoin	18
II.1.2	L'existant : un défi de taille et de structure de la donnée	18
II.1.3	Migration et regroupement familial	18
II.1.4	Refonte logicielle et application de gestion ECM	19
II.1.5	Contraintes d'interopérabilité	19
II.2	Migration et regroupement familial	19
II.2.1	Choix technologiques et rigueur logicielle	20
II.2.2	Architecture transactionnelle et asynchrone	20
II.2.3	Stratégies de résolution des conflits	21
II.2.4	Ingénierie du déploiement et conteneurisation	22
II.2.5	Limites du modèle de données plat	22
II.3	Modélisation et normalisation de la base ECM	22
II.3.1	Du modèle plat au modèle relationnel	22
II.3.2	Schéma de la base de données	23
II.3.3	La chaîne de traitement des thésaurus	25
II.3.4	La commande d'import principale	26
II.4	Application principale : architecture et réalisation	26
II.4.1	Architecture technique et paradigme frontal	26
II.4.2	Modélisation des droits et rôles	26
II.4.3	Rigueur logicielle et Intégration Continue	27
II.4.4	Implémentation des règles métiers et sécurité	29
II.4.5	Défis techniques des modules fonctionnels	29
II.5	Bilan technique et perspectives	32
III	PIAWEB : Une histoire de DevOps	34
III.1	Anatomie d'une infrastructure industrialisée	34
III.1.1	Architecture de l'environnement DevOps et de l'infrastructure	34
III.1.2	Intégration Continue et Déploiement Continu (CI/CD)	35

III.1.3	Gestion des environnements et stratégie de branche	35
III.1.4	Un anti-pattern relevé : la stratégie de branches du dépôt <code>docker</code>	36
III.1.5	Le processus de livraison et de montée de version	37
III.2	Première livraison, première leçon	38
III.2.1	Le bogue	38
III.2.2	Appareillage sur lest	38
IV	Autres projets en cours	40
IV.1	Automatisation de la qualification	40
V	Conclusion	41
V.1	À propos de PHP et Symfony : une cacophonie harmonisée	41
V.1.1	Une histoire de typage progressif	41
V.1.2	Un parallèle avec l'écosystème JavaScript	41
V.1.3	Symfony face à Spring Boot	41
V.1.4	Le défi de la performance de l'outillage	42
V.2	Enrichissement personnel	42
	Bibliographie	43
	Références logicielles	44

Table des figures

Fig. 1	Organigramme de l'entreprise	7
Fig. 2	Capture de l'écran d'accueil de <code>info.gouv.fr</code> , utilisant le DSFR	13
Fig. 3	Flux de traitement de migration et résolution de conflits	21
Fig. 4	Capture de la page de résolution de conflits	21
Fig. 5	Export Figma de l'écran de la page d'administration des thésaurus	23
Fig. 6	Diagramme UML simplifié de l'entité Soldat	23
Fig. 7	Capture de la sortie de la commande de synchronisation de tous les thésaurus	25
Fig. 8	Rôles applicatifs et étendue associée	27
Fig. 9	Export Figma de l'écran de recherche par individu avec résultats	30
Fig. 10	Export Figma de l'écran de gestion d'un site (vue du chef de secteur)	31
Fig. 11	Export Figma la modale de validation des modifications	31
Fig. 12	Export Figma de l'écran de l'écran d'export vers Mémoire des Hommes ¹ (MdH)	32
Fig. 13	Architecture physique et logique de l'infrastructure PIWEB	35
Fig. 14	Schéma de flux illustrant les étapes de la livraison	38

¹<https://memoiredeshommes.defense.gouv.fr>

I Introduction

I.1 Présentation de l'entreprise

Actimage est une entreprise de services numériques (ESN) française créée en 1995 par Christophe Megel, l'actuel PDG. Le travail de l'entreprise est divisé en 3 pôles principaux communicants: le développement, le conseil et la recherche et développement (R et D). Mon stage s'est déroulé dans le pôle développement mais j'ai eu l'occasion d'interagir avec le pôle R et D sur certains sujets. Les effectifs d'Actimage sont répartis entre 8 agences dans 5 pays. Celles avec lesquelles j'ai le plus été en contact sont Paris, Arcueil, Colmar, Strasbourg, Metz et Luxembourg.

I.1.1 Composition de l'entreprise

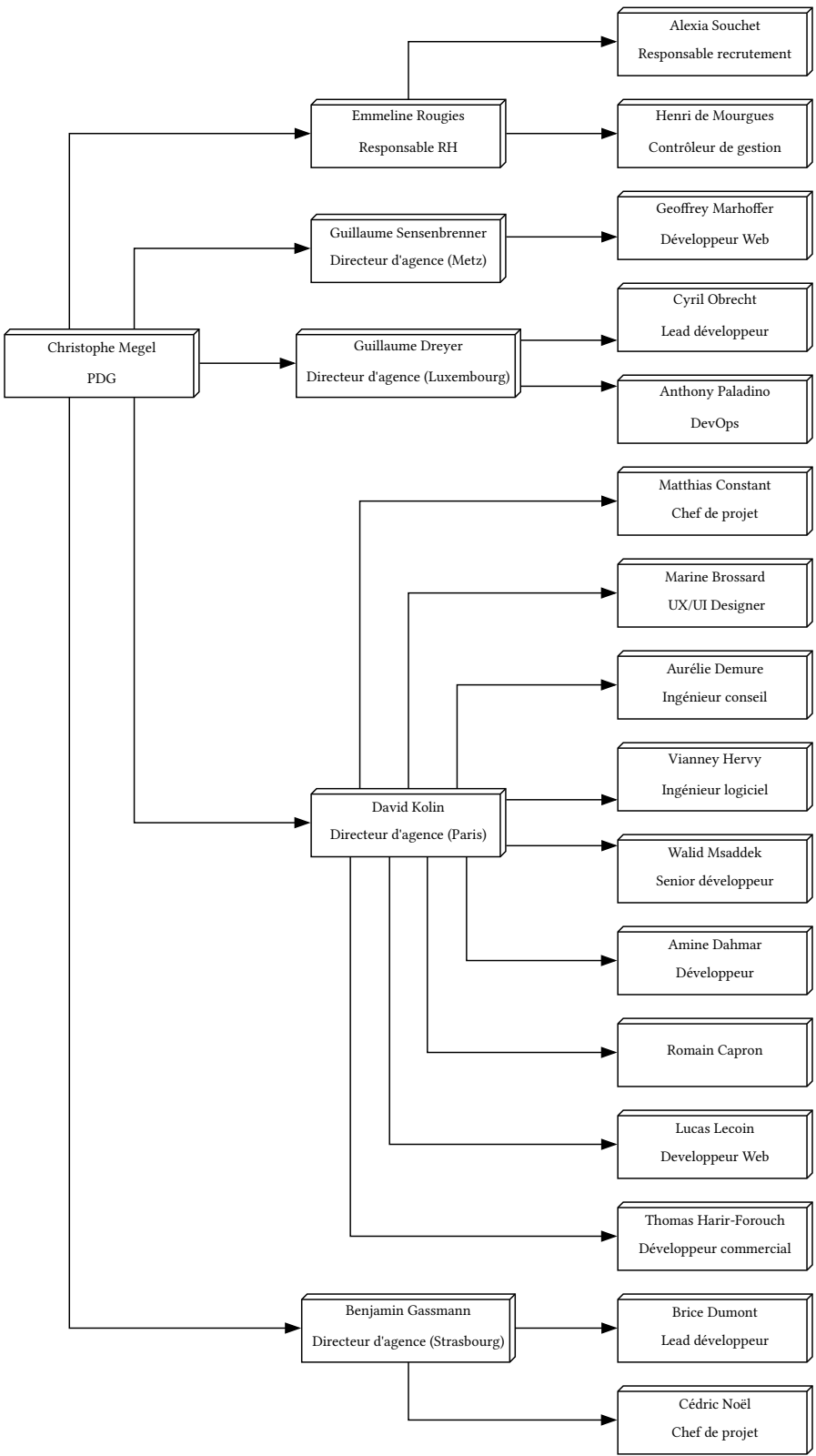


Fig. 1. – Organigramme de l'entreprise

²David a quitté Actimage le 31 Aout 2027, juste avant la fin de mon stage.

I.1.2 Équipe rapprochée

David Kolin (DKO)

David était² directeur de l'agence Paris-Arcueil depuis Janvier 2025 et mon maître de stage. Il a une formation d'ingénieur informatique et une expérience en ESN (Capgemini) et surtout en contrats publiques, notamment acquise pendant 13 ans au Centre des monuments nationaux³ (CMN). L'une de ses missions principales était de répondre à des appel d'offre (AO), mettant à profit à la fois son expérience dans cette tâche particulière et son expertise technique, lui permettant de répondre en connaissance des besoins clients et des contraintes techniques précises. C'est cette double compétence qui m'a de suite plu. Avoir un supérieur éclairé sur ce qui est possible, ce qui est difficile de développer permet d'éviter l'écueil classique de la déconnexion entre la phase d'avant-vente - impliquant la réponse à l'AO, le cahier des charges et le chiffrage - et la réalité concrète du développement. Les engagements et promesses commerciales restent ainsi en parfaite adéquation avec la faisabilité technique du terrain. Cela, bien sûr, n'empêche pas la réévaluation post-signature du coût du contrat. Certains besoins trop peu développés dans le cahier des charges peuvent s'avérer plus coûteux que prévu et causer des frais additionnels imprévus.

Matthias Constant (MCO)

Matthias est chef de projet et consultant IT chez Actimage depuis 2016. Bien qu'il n'ait qu'une formation en école de management, sa carrière dans le domaine du numérique lui a permis d'acquérir des compétences que l'on ne prêterait pas à une personne d'origine non-technique. Pareillement à David, il a la capacité de mesurer l'effort technique nécessaire au développement d'une fonctionnalité demandée par le client. Développeur de loin, il lui est courant d'utiliser des modèles de langage pour réaliser une preuve de concept (POC) permettant une meilleure estimation du coût et temps de travail nécessaire sur un projet. Ces POC sont parfois repris tels quels par les développeurs au début du projet en tant que base.

Marine Brossard (MBR)

Marine est Designer UX/UI et cheffe de projet chez Actimage depuis mars 2025. Son outil principal de maquettage est Figma [7], qui permet notamment de définir des composants et des règles de style correspondant directement à des propriétés CSS et des classes TailwindCSS. Au-delà de sa maîtrise technique, sa capacité à structurer les éléments graphiques offre une première décomposition fonctionnelle et technique pour des projets qui pourraient paraître denses et complexes.

Aurélie Demure (ADE)

Aurélie est en stage de fin d'étude chez Actimage depuis mars 2027 en tant qu'ingénieure conseil. Elle prépare son double diplôme d'ingénieur de TELECOM Nancy⁴ et de l'Institut Mines-Télécom Business School⁵ (IMT-BS). Ses missions incluent la qualification d'AO, le sourcing de candidats à recruter⁶ ainsi que la rédaction de spécifications fonctionnelles détaillées (SFD). Son expertise technique au niveau d'un ingénieur du numérique lui a aussi permis de s'impliquer dans le développement d'une fonctionnalité du projet Office national des combattants et des victimes de guerre⁷ (ONaCVG).

Romain Capron (RCO)

Romain est ingénieur diplômé de Polytech Sorbonne⁸ en Mathématiques appliquées et Informatique. Ex-stagiaire, il appartient depuis juin 2026 au pôle R et D d'Actimage au sein duquel il développe un outil permettant d'automatiser sinon d'optimiser la qualification d'AO.

³<https://www.monuments-nationaux.fr>

⁴<https://telecomnancy.univ-lorraine.fr>

⁵<https://www.imt-bs.eu>

⁶La proposition de stage qu'Actimage m'a faite parvenir vient notamment d'un sourcing

⁷<https://www.onac-vg.fr>

⁸<https://www.polytech.sorbonne-universite.fr>

Thomas est en contrat d’alternance chez Actimage depuis septembre 2025. Il suit une formation de développeur commercial à Audencia⁹. Ses missions principales incluent la prospection commerciale, la qualification d’AO, la réponse à AO ainsi que le recrutement (sourcing et entretiens).

I.2 Contexte du stage

I.3 Cycle de vie d’un projet

I.3.1 Avant-vente et commercialisation

La phase d’avant-vente constitue le point d’entrée critique de tout projet au sein d’une ESN. Pour une entreprise dont une part significative de l’activité repose sur les marchés publics, cette étape conditionne non seulement la viabilité financière de la structure, mais également son positionnement stratégique à long terme.

Prospection commerciale et veille stratégique

L’acquisition de nouveaux projets débute par une démarche proactive de prospection commerciale et de veille stratégique. Des collaborateurs dédiés, tels que Thomas au sein de l’agence, scrutent quotidiennement le marché et les plateformes de publication de marchés publics. Pour Actimage, c’est notamment le Portail des marchés publics¹⁰. L’objectif premier est l’identification d’AO entrant en résonance avec le savoir-faire technologique de l’entreprise, ses références passées et ses ambitions de développement. Cette veille constante permet d’alimenter le carnet d’opportunités de l’agence tout en maintenant une connaissance affûtée des besoins évolutifs des administrations et des grandes entreprises.

Qualification du marché

Une fois un AO repéré, il fait l’objet d’une phase de qualification rigoureuse. Cette étape, à laquelle contribuent activement des profils hybrides comme Aurélie ou Thomas, agit comme un filtre décisionnel essentiel. Il s’agit d’évaluer la pertinence d’un positionnement de l’entreprise sur le marché identifié en analysant une grille de critères déterminants : la disponibilité immédiate et future des ressources en interne, l’adéquation des compétences techniques requises avec l’expertise réelle de l’agence, ainsi que la solidité du modèle économique proposé. Une qualification minutieuse et lucide est indispensable pour éviter la mobilisation chronophage et coûteuse des équipes sur l’élaboration de réponses aux chances de succès trop minces.

Preuve de concept et chiffrage

Si l’opportunité est qualifiée positivement, le projet entre dans une phase de projection et d’évaluation technique. Afin de proposer un chiffrage précis et de mesurer l’effort de développement nécessaire, des preuves de concept (POC) sont fréquemment réalisées. Des chefs de projet expérimentés, à l’image de Matthias, s’appuient notamment sur des modèles de langage pour générer rapidement des prototypes fonctionnels ou défricher de nouvelles piles technologiques. Cette approche exploratoire permet de lever les incertitudes techniques, d’identifier en amont les éventuels défis architecturaux et de consolider l’estimation budgétaire en confrontant la théorie du besoin à une première implémentation pratique.

Stratégie de réponse et alignement technico-commercial

L’élaboration de la réponse à l’appel d’offres exige une synergie totale entre la vision commerciale et le pragmatisme technique. Sous l’impulsion de la direction de l’agence, représentée par David, le mémoire technique et l’offre financière sont construits en parallèle et en concertation. C’est ici que la double compétence de la direction prend tout son sens : elle garantit que les promesses fonctionnelles et les délais annoncés restent en parfaite adéquation avec la faisabilité sur le terrain. Cet alignement technico-commercial est fondamental

⁹<https://www.audencia.com>

¹⁰<https://www.marches-publics.gouv.fr>

pour éviter l'écueil classique du cahier des charges irréalisable ou sous-évalué, sécurisant ainsi la rentabilité du projet et la sérénité des futures équipes de développement.

Signature et réévaluation

Enfin, en cas d'attribution du marché, la phase d'avant-vente s'achève par la contractualisation, officialisant la transition du statut de prospect à celui de client. Cette ultime étape administrative peut néanmoins s'accompagner d'une phase de réévaluation budgétaire post-signature. En effet, les premières réunions d'immersion et la confrontation du cahier des charges initial aux contraintes concrètes du client révèlent parfois des zones d'ombre, des oublis ou des besoins implicites plus complexes que prévu, nécessitant un ajustement contractuel transparent avant le lancement effectif de la chaîne de production.

I.3.2 Pilotage Itératif et Outillage de Suivi

La traduction des exigences d'un appel d'offres en un produit logiciel fonctionnel requiert une méthodologie de gestion de projet alliant rigueur structurelle et souplesse d'exécution. Au sein d'Actimage, cette dynamique s'appuie sur un pilotage itératif, soutenu par une communication transparente et régulière avec le client.

Initialisation et évolution continue du périmètre

Le lancement de la phase de développement se matérialise par la création d'une première série de tickets par le chef de projet, reflétant les exigences initiales extraites du cahier des charges et des premières maquettes. Toutefois, loin d'adopter un modèle purement séquentiel et figé de type « cycle en V », la méthodologie privilégie l'agilité et l'adaptation. Au gré des ateliers de co-conception qui se poursuivent en parallèle de la production du code, le périmètre fonctionnel s'affine continuellement. Les tickets initiaux sont ainsi régulièrement réécrits, subdivisés pour en réduire la complexité, ou complétés par de nouvelles tâches afin de répondre avec justesse aux besoins émergents du client.

Points hebdomadaires et relation client

La cohésion du projet et l'alignement des visions sont maintenus grâce à un rituel de synchronisation hebdomadaire réunissant le client, le chef de projet et, de manière stratégique, un ou plusieurs développeurs. Ces points d'étape réguliers sont l'occasion de présenter l'état d'avancement concret du produit à travers des démonstrations des dernières fonctionnalités implémentées. L'implication directe d'un développeur lors de ces échanges s'avère particulièrement bénéfique. D'une part, elle permet au client de mettre un visage sur l'acteur de la réalisation de son outil, instaurant ainsi une proximité et un fort climat de confiance. D'autre part, cette désintermédiation facilite la remontée proactive d'obstacles techniques ou de questionnements fonctionnels bloquants. Le client peut alors y répondre immédiatement en séance, ou consigner ces points pour apporter des précisions ultérieurement par courriel ou lors de la réunion suivante.

Suivi opérationnel et traçabilité technique (GitLab)

Pour orchestrer finement cette production itérative, l'équipe s'appuie sur l'instance GitLab auto-hébergée, l'appui principal du développement logiciel. Cet outil centralise les tickets techniques qui se distinguent par leur exhaustivité, documentant les règles métiers, les contraintes d'architecture et les critères d'acceptation. Ces tickets sont directement couplés aux branches de développement Git. Cette plateforme héberge également l'ensemble des fils de discussion relatifs à l'implémentation, garantissant ainsi une traçabilité totale entre le besoin fonctionnel initial, les arbitrages techniques décidés en équipe, et l'historique du code source.

Imputation et pilotage de la rentabilité (Redmine)

En parallèle de ce suivi purement opérationnel, une séparation de l'outillage est instaurée pour répondre aux impératifs de gestion administrative et financière inhérents au fonctionnement d'une ESN. Actimage déploie à cet effet un outil Redmine interne, principalement dédié à la saisie des temps de travail (imputations) par les différents collaborateurs mobilisés. Cette plateforme héberge des macro-tickets de projet, volontairement moins détaillés et dont la mise à jour est moins fréquente que sur GitLab. Leur vocation est avant tout

budgétaire et analytique : lors de leur clôture, ces tickets agrègent les heures consommées par l'ensemble des acteurs (designers, chefs de projet, développeurs) et permettent à la chefferie de projet d'évaluer avec une grande précision le coût de revient réel de chaque fonctionnalité livrée.

I.3.3 Développement et Assurance Qualité

Une fois les besoins fonctionnels figés et spécifiés, le projet entre dans sa phase de réalisation technique (souvent désignée sous le terme de « Build »). Au sein de l'agence, cette phase est encadrée par des processus stricts visant à garantir non seulement la vélocité de l'équipe, mais surtout la fiabilité, la sécurité et la maintenabilité du produit final.

Découpage technique et répartition des développements

La transition vers la production de code est amorcée par un minutieux travail d'architecture et de découpage technique. Le *lead developer*, un rôle assuré par exemple par Brice sur le projet de l'ONaCVG, est chargé de fragmenter les spécifications fonctionnelles détaillées en unités logiques et en tickets techniques indépendants. Ces tâches sont ensuite distribuées et assignées aux différents développeurs de l'équipe (tels qu'Amine ou moi-même). Cette répartition s'effectue de manière stratégique, en prenant en compte les domaines d'expertise spécifiques de chacun, la capacité de production et les disponibilités du moment au sein de l'agence.

Environnements d'exécution locaux et conteneurisation

Afin de garantir la reproductibilité du code et de prévenir les classiques conflits de dépendances, le développement s'effectue de prime abord au sein d'environnements locaux strictement isolés. Les différents composants des projets sont conteneurisés et orchestrés via des outils comme Docker. Cette virtualisation légère permet à chaque développeur d'exécuter une pile technologique identique (bases de données, serveurs web, workers asynchrones) indépendamment de son système d'exploitation hôte, assurant ainsi que le code produit réagira de manière prédictible de la machine du développeur jusqu'aux serveurs de production du client.

Intégration Continue et rigueur logicielle

La garantie d'une haute qualité logicielle est automatisée grâce à la mise en place d'une chaîne d'intégration continue (par exemple via des *pipelines* Jenkins ou GitLab CI) particulièrement exigeante. Chaque demande de fusion de branche Git initiée par un développeur déclenche l'exécution de processus de validation bloquants. L'architecture s'appuie sur une analyse statique stricte, pilotée par des outils comme PHPStan poussés à leur niveau d'exigence maximal, ce qui prévient les erreurs d'exécution en imposant un typage fort et robuste. Parallèlement, des outils d'analyse syntaxique assurent un formatage automatisé du code pour maintenir une homogénéité parfaite à l'échelle de l'équipe, tandis que des utilitaires comme Rector suggèrent des refactorisations architecturales pertinentes.

Sécurité automatisée et validation par les tests

Outre l'application de ces standards de qualité, le code soumis subit un audit de sécurité automatisé avant toute intégration. Des outils de balayage tels que Gitleaks parcourent l'historique des modifications à la recherche d'expressions régulières correspondant à des secrets de configuration ou des clés d'API, empêchant ainsi toute fuite de données sensibles dans le code source. Enfin, la solidité fonctionnelle est prouvée par l'exécution de tests automatisés. Couvrant à la fois des périmètres unitaires et fonctionnels via des cadriciels comme PHPUnit, ces tests sont souvent adossés à des mécanismes de bases de données transactionnelles (via des paquets dédiés isolant l'état de la base à chaque test). Cette isolation garantit que chaque composant réagit précisément tel qu'attendu par les spécifications, sans générer de régressions.

La dernière étape, précédant l'intégration définitive du code sur la branche principale du projet, fait appel à l'expertise humaine. Si l'intégration continue s'assure de l'intégrité syntaxique et de la validation des tests, chaque demande de fusion fait l'objet d'une revue par le *lead developer*. Cette étape est cruciale pour valider les choix algorithmiques, s'assurer que la logique métier implémentée est la réponse la plus élégante aux besoins du client, et favoriser la diffusion des bonnes pratiques et de la connaissance technique au sein du pôle.

Dans certains cas très exceptionnels tels que pendant les premières semaines de développement, le cycle est allégé, les *pipelines* CI/CD sont non-bloquantes et les demandes de fusion peuvent être auto-validées.

1.3.4 Déploiement, Recette et Maintenance

L'aboutissement de la phase de développement marque le début d'un processus de livraison hautement sécurisé. Transférer le code depuis l'ordinateur d'un développeur jusqu'aux serveurs finaux exige une maîtrise parfaite de l'infrastructure afin de garantir la stabilité du produit, la sécurité des données et la pérennité de l'application sur le long terme.

Montée en environnements et figeage des livrables

Le cycle de vie d'une version logicielle s'articule autour d'une progression maîtrisée à travers une cascade d'environnements distincts : Développement, Intégration, Recette, Pré-production et Production. Si les premiers environnements favorisent la vélocité en montant dynamiquement le code source via des volumes, le passage en recette marque un changement strict de paradigme. Le code applicatif est alors compilé de manière statique et encapsulé en dur au sein d'images Docker immuables, qui sont ensuite poussées vers le Harbor (registre privé d'Actimage). Ce figeage technologique est fondamental : il garantit mathématiquement que l'image logicielle testée par le client sera strictement identique à celle qui sera *in fine* déployée en production.

Phase de Recette Client

Une fois les livrables sécurisés et conteneurisés, l'application est propulsée sur l'environnement de recette (rec), un espace de démonstration dédié, isolé, et mis à la disposition exclusive du client. Cette phase critique permet aux interlocuteurs métiers et aux utilisateurs finaux de mener leurs propres campagnes de tests afin de vérifier la conformité stricte du produit livré vis-à-vis des SFD. L'utilisation d'images Docker prêtes pour la production élimine le risque d'anomalies liées à des différences de configuration d'environnement, garantissant ainsi au client une expérience de recette parfaitement authentique et représentative du produit final.

Mise en Production

Le déploiement final, ou Mise en Production, est une opération délicate qui ne tolère aucune improvisation. Pour éviter toute corruption de données ou interruption de service prolongée, cette montée de version obéit à une chorégraphie procédurale immuable. Elle débute systématiquement par l'arrêt ordonné et propre des services applicatifs en cours d'exécution. Immédiatement après, une sauvegarde complète de la base de données (généralement via un export compressé) est réalisée pour sanctuariser un point de restauration immédiat en cas de défaillance. S'ensuivent la modification des variables de configuration d'environnement, l'exécution automatisée des scripts de migration de schéma (orchestrée par des outils éphémères dédiés comme Flyway), et le redémarrage séquentiel des conteneurs. Une attention particulière est portée à l'ordre de relance, le composant dorsal (backend) devant être pleinement opérationnel avant d'activer le composant frontal (frontend) pour éviter les erreurs de connexion asynchrone.

Tierce Maintenance Applicative

La validation d'une mise en production majeure ne signe pas l'achèvement du projet, mais sa transition vers une phase de tierce maintenance applicative (TMA). Les équipes de développement, bien que réduites, continuent d'intervenir sur le code pour diagnostiquer et corriger les anomalies résiduelles remontées par les

utilisateurs via les outils de billetterie, pour implémenter des évolutions fonctionnelles mineures, ou encore pour ajuster les flux d'interopérabilité avec les systèmes tiers. Ce suivi à long terme, illustré par exemple par le projet PIAWEB, permet d'absorber les évolutions métiers du client et de garantir la viabilité de l'application face à l'épreuve du temps.

I.3.5 Réalisations

DSFR

L'expertise d'Actimage en développement brille tout particulièrement sur les marchés publics. L'entreprise a notamment participé à la conception du Système de Design de l'État Français¹¹ (DSFR). Le DSFR regroupe un ensemble de règles et de composants réutilisables pour les interfaces officielles des sites en .gouv.fr. Il permet à l'État d'offrir des services numériques simples, accessibles et reconnaissables. C'est notamment celui que vous retrouvez sur impots.gouv.fr, ants.gouv.fr et sante.gouv.fr.

Ce système de design est conçu pour être agnostique et modulaire. Il se décline sous plusieurs formats afin de couvrir tout le cycle de vie d'un projet de la phase de conception, au prototypage, à l'implémentation technique. Il existe notamment une librairie Figma, un socle de base en HTML/CSS/JS natif à travers le paquet `@gouvfr/dsfr` ainsi que des portages développés par la communauté tels que `@codegouvfr/react-dsfr` (React), `@gouvminint/vue-dsfr` (Vue), `ngx-dsfr` (Angular), `django-dsfr` (Django) et `drupal/ui_suite_dsfr` (Drupal).



Fig. 2. – Capture de l'écran d'accueil de info.gouv.fr, utilisant le DSFR

Site officiel du Gouvernement

L'un des sites majeurs du gouvernement est info.gouv.fr. C'est aussi un des projets principaux d'Actimage qui en réalise le développement côté serveur et une partie du développement côté client. Plusieurs développeurs travaillent à temps plein sur ce projet, dont même certains physiquement au Service d'information du Gouvernement¹² (SIG).

BDnf

Actimage est également l'entreprise derrière BDnF¹³, un outil ludo-éducatif de création de bandes dessinées et de récits multimédias développé pour le compte de la Bibliothèque nationale de France¹⁴ (BnF). Destinée

¹¹<https://www.systeme-de-design.gouv.fr>

¹²<https://www.info.gouv.fr/organisation/service-d-information-du-gouvernement-sig>

¹³<https://bdf.bnf.fr>

¹⁴<https://www.bnf.fr>

principalement au milieu scolaire, l'application se connecte à la bibliothèque numérique Gallica via un module d'import permettant aux utilisateurs d'intégrer directement des corpus d'images d'archives dans leurs projets.

Techniquement, l'application a été développée de manière multi-support avec le moteur Unity 3D. L'architecture logicielle repose sur un noyau commun partagé entre les environnements de bureau (macOS, Windows) et tablettes (Android, iOS), tout en implémentant des fonctionnalités spécifiques adaptées aux contraintes des versions mobiles. Afin d'accélérer le développement des nouvelles fonctionnalités et de garantir la cohérence des interfaces sur toutes ces plateformes, l'équipe s'est appuyée sur la mise en place d'un système de design strict. Enfin, la conception a fait l'objet de tests d'utilisabilité itératifs menés directement auprès d'élèves pour valider l'ergonomie.

Hol'Autisme

Actimage s'inscrit fortement dans l'innovation et la R et D avec des projets à fort impact sociétal à l'image de Hol'Autisme¹⁵. Ce projet novateur du pôle R et D propose le premier catalogue d'applications en réalité mixte destiné à aider les enfants et adolescents atteints de troubles du spectre autistique à développer leurs compétences sociales. Développée notamment avec le moteur Unity pour le casque HoloLens, la solution permet de simuler des situations du public ou du quotidien dans un environnement interactif et contrôlé. Le but est d'aider les patients à appréhender les codes sociaux et à gagner progressivement en autonomie sans subir l'angoisse du monde réel.

L'expertise technologique du projet va bien au-delà de la simple réalité mixte : le système intègre un bracelet connecté permettant de mesurer le niveau d'anxiété de l'apprenant en temps réel, couplé à une plateforme web de contrôle et de suivi. Grâce à l'analyse de données et à des outils statistiques avancés, le personnel médico-éducatif peut analyser finement les sessions. La pertinence de ce dispositif global, dont la première preuve de concept s'intitule PopBalloons, a d'ailleurs été saluée par l'écosystème technologique, le projet étant lauréat des concours French IOT 2017 et Futur.e.s 2018.

I.4 Poste de travail et outillage de développement

I.4.1 Environnement d'exécution et philosophie de travail

Au sein d'Actimage, le matériel mis à ma disposition était un poste opérant sous Windows 11. Afin de retrouver un environnement de développement familier, performant et conforme à la philosophie Unix qui guide mon flux de travail quotidien, j'ai fait le choix d'exploiter le sous-système Windows pour Linux [8]. J'y ai déployé une distribution Arch Linux. Cette approche m'a permis de répliquer quasi à l'identique l'environnement modulaire que j'utilise à titre personnel, tout en m'affranchissant des limitations inhérentes au système d'exploitation hôte pour le développement d'applications web.

I.4.2 Outils collaboratifs et centralisation de la connaissance

L'environnement de travail d'un ingénieur logiciel ne se limite pas à son éditeur de code et à son terminal. La dimension collaborative et la gestion de la connaissance sont des composantes tout aussi vitales, particulièrement au sein d'une ESN où de multiples acteurs (clients, chefs de projet, designers, développeurs) interagissent quotidiennement. Pour orchestrer cette communication, Actimage s'appuie principalement sur la suite Microsoft.

Le flux de communication synchrone était assuré par Microsoft Teams. Bien que cet outil ne bénéficiât d'aucune intégration automatisée avec notre chaîne d'intégration continue (comme des alertes webhooks liées à Jenkins ou GitLab), il constituait le centre névralgique de la vie d'équipe. C'est sur cette plateforme que se tenaient nos réunions quotidiennes de synchronisation (*daily stand-ups*). L'application était structurée en différents canaux dédiés aux projets, servant d'espaces d'échanges informels pour partager des extraits de

¹⁵<https://www.holautisme.com>

code, soumettre des idées d'architecture, formuler des remarques ou solliciter de l'aide auprès de développeurs plus expérimentés comme Brice ou Matthias.

Pour la communication asynchrone et formelle, Microsoft Outlook était de rigueur. Ce canal était privilégié pour les échanges directs avec les clients, souvent organisés via des listes de diffusion. La boîte de réception agissait également comme un agrégateur d'événements : j'y recevais les rappels de réunions, les annonces internes de l'entreprise, mais surtout les notifications automatisées générées par les outils de billetterie (*ticketing*) des clients. Ces alertes me permettaient de suivre en temps réel la progression des tickets d'anomalie ou l'évolution d'un fil de discussion lié à une spécification fonctionnelle.

Concernant la gestion documentaire, une dichotomie marquée existait entre les aspects « métiers » et les aspects purement « techniques » des projets. Toute la documentation fonctionnelle et contractuelle était centralisée sur SharePoint. On y retrouvait une arborescence dense composée de documents Microsoft Word pour les spécifications, des notes de restitution d'ateliers clients, ainsi que les volumineux fichiers CSV servant de thésaurus pour l'application de l'ONaCVG.

Cependant, l'expérience utilisateur offerte par l'interface web de SharePoint contrastait fortement avec la vélocité de mon environnement de développement sous Linux. Naviguer au sein de ces dossiers imbriqués s'avérait particulièrement laborieux en comparaison de la fluidité offerte par des outils de navigation en ligne de commande basés sur des recherches floues, tels que *fzf* ou *zoxide*, que j'utilise quotidiennement dans mon terminal.

Par ailleurs, la capitalisation de la connaissance technique souffrait d'un certain morcellement. Si les règles métiers résidaient sur SharePoint, la documentation technique était éparpillée entre les dépôts Git (via les fichiers *README.md*), la plateforme Bookstack de l'entreprise, et un Wiki interne distinct. Cette fragmentation imposait une gymnastique mentale constante pour retrouver la bonne information au bon endroit.

1.4.3 Inspection web et débogage dynamique

Si le terminal et Neovim constituent mon espace de création logique, le navigateur web représente l'environnement d'exécution final, le véritable « compilateur visuel » de mon travail. Au sein d'Actimage, bien que certains collaborateurs aient opté pour Mozilla Firefox ou Opera, j'ai fait le choix de conserver Google Chrome. Cette décision était principalement motivée par une continuité d'usage avec mon environnement personnel, mais aussi pour sa rapidité d'exécution et la modernité de son moteur.

Ma configuration du navigateur est restée minimaliste. La seule extension véritablement indispensable à mon flux de travail était Bitwarden, configurée pour se synchroniser avec l'Actipass, l'instance auto-hébergée du gestionnaire de mots de passe d'Actimage, garantissant un accès sécurisé aux différents environnements de développement et de pré-production. Avec le recul, la création de profils Chrome distincts (un personnel et un professionnel) aurait été judicieuse. Par manque d'isolation de contexte, mes favoris personnels et professionnels se sont retrouvés entremêlés. Pour pallier l'absence de profils isolés par projet et éviter les conflits de sessions ou d'états, j'ai pris l'habitude de recourir massivement à la navigation privée ou aux rechargements forcés avec vidage du cache afin de simuler le comportement d'un nouvel utilisateur vierge de tout historique.

Dans l'écosystème Symfony, une grande partie du débogage est facilitée par l'excellent profileur Symfony, une barre d'outils injectée en bas de page offrant une visibilité totale sur le cycle de vie de la requête HTTP (requêtes SQL exécutées, temps de réponse, formulaires soumis, événements déclenchés). Néanmoins, le profileur atteint ses limites dès lors qu'il s'agit d'inspecter le comportement du code exécuté côté client. C'est ici que les outils de développement intégrés à Chrome (*DevTools*) prenaient le relais.

Je sollicitais les *DevTools* pour plusieurs cas d'usage bien précis :

L'onglet Éléments (Elements)

Il s'est avéré particulièrement redoutable pour le développement de l'interface utilisateur. Utilisant le cadriciel CSS utilitaire Tailwind, je pouvais manipuler les attributs de classe des éléments HTML à la volée directement

dans le DOM. Cette méthode permet de valider instantanément un comportement visuel ou un ajustement responsif dans le navigateur, avant d’aller inscrire la classe correspondante en dur dans les gabarits Twig.

L’onglet Sources

Je le consultais régulièrement pour m’assurer que le navigateur avait bien récupéré les dernières versions compilées de mes scripts JavaScript et feuilles de style, un point de vérification essentiel lors de l’utilisation du composant Symfony AssetMapper qui gère le versionnage des fichiers statiques.

L’onglet Réseau (Network)

Bien que le profileur Symfony permette d’analyser le temps des requêtes, l’onglet Réseau de Chrome était très pratique pour visualiser les requêtes asynchrones en cascade, notamment lors de l’ingestion de lourdes charges de données ou lors des appels API du formulaire d’inspection de l’ONaCVG.

La Console

Elle venait combler une lacune majeure du profileur Symfony : l’absence d’agrégation des erreurs JavaScript. La console était mon outil de diagnostic principal pour traquer les avertissements, lire les erreurs remontées par les contrôleurs Stimulus (Symfony UX), ou exécuter rapidement des requêtes exploratoires sur des objets du DOM.

Enfin, concernant le débogage côté serveur, l’outil Xdebug était bien configuré et présent dans la pile Docker de l’application ONaCVG. Toutefois, je n’en ai eu qu’un usage extrêmement marginal. La combinaison d’une analyse statique très stricte (PHPStan), garantissant la cohérence des types et de la logique structurale en amont, couplée à la richesse d’informations délivrées par le profileur Symfony et aux tests unitaires, permettait d’identifier l’origine des anomalies sans avoir à recourir à l’exécution pas-à-pas offerte par un débogueur traditionnel.

I.4.4 L’éditeur de code: le choix de Neovim

Face à la complexité de l’écosystème PHP/Symfony, j’ai dans un premier temps évalué l’utilisation d’un environnement de développement intégré (IDE) classique, tel que PhpStorm [9]. Bien que cet outil propose des intégrations natives puissantes, sa lourdeur d’exécution et son manque de flexibilité (même pallié par l’extension IdeaVim [10]) ont constitué des freins à ma productivité. J’ai donc réintégré Neovim [11] comme éditeur principal.

Le langage PHP étant interprété et historiquement peu strict sur le typage, il ne bénéficie pas nativement des mêmes garanties à l’écriture qu’un langage compilé. Pour pallier cela et atteindre une rigueur de développement similaire à celle qu’offre TypeScript dans l’écosystème JavaScript, j’ai dû configurer un outillage robuste basé sur le Language Server Protocol (LSP) [12].

- J’ai principalement utilisé Phpactor [13] etintelephense [14] pour l’autocomplétion et l’analyse statique.
- J’ai également expérimenté avec PHPantom [15] un serveur de langage récent développé en Rust [16], offrant des performances d’exécution nettement supérieures.
- L’intégration récente d’un serveur de langage dédié spécifiquement à Symfony [17] est venue parfaire cette configuration, me permettant d’avoir un retour intelligent sur les spécificités du cadriceil.

L’analyse de la qualité du code est restée isolée localement pour chaque projet, tout en étant exploitée par les serveurs de langage pour remonter les erreurs directement dans l’éditeur.

I.4.5 Contributions aux logiciels libres et outillage sur mesure

Constatant que certains outils de l’écosystème manquaient de maturité par rapport à d’autres langages, j’ai adopté une démarche proactive en contribuant à des projets à code source ouvert. J’ai notamment signalé et résolu des anomalies sur Twiggy [18] (un serveur de langage pour le moteur de gabarits Twig) et amélioré la grammaire Tree-sitter [19] de Twig, utilisée par Neovim pour l’analyse syntaxique. Cette implication m’a

également amené à échanger avec Fabien Potencier, créateur de Symfony, autour du développement de leur nouveau serveur de langage.

Pour optimiser la navigation dans les bases de code PHP, j'ai également développé des requêtes Tree-sitter personnalisées pour le greffon vim-matchup [20]. Celles-ci offrent une navigation syntaxique avancée en définissant des portées spécifiques pour les balises et les structures de contrôle du langage. Il devient ainsi possible de naviguer intelligemment entre les différentes clauses d'une condition ou d'une boucle, de parcourir aisément les blocs complexes (tels que `switch`, `match` ou `try catch`), et de sauter instantanément de la signature d'une fonction à ses instructions de retour.

I.4.6 Le terminal comme espace de travail unifié

La reproduction de mon environnement s'arrêtant aux frontières du WSL et donc du terminal, j'ai optimisé ce dernier pour limiter au maximum l'usage de la souris et la friction liée aux changements de contexte. Le multiplexeur `tmux` [21] a été central dans cette démarche en me permettant de gérer de multiples invites de commande au sein d'une même fenêtre.

Afin de fluidifier mon flux de travail, j'ai enrichi cet espace de scripts et de raccourcis sur mesure. Ces personnalisations me permettent d'invoquer des interfaces interactives telles que `lazygit` [22] et `lazydocker` [23] sous forme de fenêtres superposées sans jamais quitter mon contexte d'édition, mais également d'interagir nativement au clavier avec des liens hypertextes ou d'accéder instantanément à l'interface web du dépôt Git du projet courant.

I.4.7 Conteneurisation et exécution locale

L'ensemble des projets, tels que PIAWEB, s'exécutaient au sein de conteneurs Docker [24]. Après une première phase d'utilisation de Docker Desktop pour Windows, j'ai rapidement constaté un manque de granularité et une interface graphique ralentissant mon flux de travail.

J'ai par conséquent migré vers une installation native du démon Docker exclusivement au sein de WSL. Ce choix m'a garanti un contrôle absolu sur les ressources et les conteneurs, pilotables intégralement en ligne de commande ou via l'interface terminale de `lazydocker`, en parfaite adéquation avec le reste de mon outillage.

I.4.8 Évolution du poste de travail

Travaillant pour la première fois dans un contexte extra-scolaire et extra-personnel pendant aussi longtemps, je trouve encore chaque semaine des points de friction, des tâches répétitives à optimiser ou automatiser. Mon poste de travail est en évolution constante, s'adaptant aux besoins de mon environnement de travail et de mes projets.

II Projet ONaCVG

Le projet principal sur lequel j'ai eu l'occasion de travailler tout au long des 6 mois de stage est une application web à destination de l'ONaCVG. **TODO: étoffer**

II.1 Introduction

II.1.1 Présentation du client et genèse du besoin

L'ONaCVG est un établissement public administratif français placé sous la tutelle du ministère des Armées et des Anciens Combattants [3]. Fondé en 1916, cet organisme a pour vocation d'assurer des missions de reconnaissance, de réparation, de solidarité et de mémoire envers les combattants, les anciens combattants et les victimes de guerre [3]. L'Office opère au bénéfice d'environ 1,81 million de ressortissants (selon des estimations de 2023) à travers un réseau de services de proximité et s'impose comme l'opérateur majeur de la politique mémorielle du ministère des Armées [3].

Parmi ses prérogatives, l'ONaCVG exerce une compétence juridique spécifique en matière de sépultures militaires, un domaine encadré par le Code des pensions militaires d'invalidité et des victimes de guerre (CPMIVG) [3]. L'institution est explicitement chargée de la mise en œuvre de l'entretien, de la rénovation et de la valorisation des sépultures de guerre [3]. En effet, la loi pose le principe d'une sépulture perpétuelle pour les militaires déclarés « Mort pour la France », qu'ils reposent au sein de nécropoles nationales ou de carrés militaires communaux, et dont l'entretien incombe à l'État [3].

C'est dans le cadre de la gestion de ce vaste patrimoine funéraire et historique, et afin de moderniser ses outils numériques, que l'institution a lancé un AO visant à concevoir une nouvelle application centralisée de gestion des sépultures. Ce marché a été remporté par Actimage fin 2025. Le périmètre du contrat couvre la conception, le développement, la TMA ainsi que l'hébergement du futur service. L'application logicielle développée est exclusivement destinée à un usage interne, ses utilisateurs finaux étant principalement les chefs de secteur de l'ONaCVG œuvrant sur le terrain et les administrateurs du pôle état civil militaire (ECM).

II.1.2 L'existant : un défi de taille et de structure de la donnée

Le principal enjeu de ce projet réside dans l'héritage technique des données. La base de données existante (sous format MS Access [25]) recense plus de 800 000 sépultures, dont certaines remontent aux guerres napoléoniennes. Cette base historique compile une multitude d'informations : état civil militaire, nom et type du site, mentions honorifiques (telles que « Mort pour la France »), nationalité, informations de recrutement, unité militaire, ou encore causes du décès.

Cependant, le départ de la personne en charge de sa maintenance a entraîné une dégradation de l'intégrité des données, transformant la base en un document tabulaire peu rigoureux. Pour pallier ce manque d'outil centralisé, plusieurs chefs de secteur avaient dupliqué la « base mère » pour maintenir leurs données localement. Cette pratique a conduit à l'émergence de multiples « bases filles » désynchronisées, comportant des identifiants conflictuels, des doublons et des incohérences.

II.1.3 Migration et regroupement familial

La première mission de mon stage, qui s'est étendue sur un mois, a consisté à développer un outil de migration indépendant. Son objectif était de regrouper les bases filles avec la base mère en détectant les conflits et en proposant des stratégies de résolution : historisation des entrées conflictuelles ou conservation des deux versions via une renumérotation intelligente. Ce premier projet, qui fera l'objet d'une section détaillée ultérieurement, a constitué une excellente porte d'entrée pour m'approprier l'environnement technique de l'entreprise (PHP, Symfony, Doctrine, PostgreSQL, Docker) et les données de l'ONaCVG.

II.1.4 Refonte logicielle et application de gestion ECM

Une fois les données fusionnées (toujours sous un format tabulaire plat d'environ quarante colonnes), la mission principale de mon stage a pu débuter : le développement de l'application de gestion complète, structurée autour de trois grands axes fonctionnels.

Modélisation et consultation (Base ECM)

Afin d'éviter la duplication et de garantir l'intégrité future des données, une refonte complète du modèle de données a été nécessaire. Nous sommes passés d'un format plat hérité du CSV à une architecture relationnelle stricte (création d'entités distinctes pour les pays, départements, communes, grades, unités, bureaux de recrutement, etc.). Une part majeure de mon travail a été consacrée à l'élaboration de la commande d'importation, capable de transformer des données libres et peu rigoureuses en entités standardisées. Sur cette base saine, un module de consultation a été développé, offrant des interfaces de recherche avancée avec de multiples filtres pour explorer les données des soldats et des sites.

Module d'inspection en mobilité

L'ONaCVG ayant la charge de sépultures à perpétuité, les chefs de secteur doivent inspecter leurs sites (parfois plus de 300 par secteur) au moins une fois par an. J'ai participé au développement d'une interface optimisée pour tablettes permettant la saisie d'inspections sur le terrain. L'agent peut y corriger les informations de la base et évaluer l'état des infrastructures (sol, barrières, stèles, plaques). Ces relevés alimentent ensuite un algorithme de calcul estimant les coûts de restauration pour les tombes et sites concernés.

Administration et flux de validation

Le troisième volet de l'application concerne les administrateurs ECM. Pour garantir la qualité de la base de données sur le long terme, un flux de travail (workflow) a été mis en place. Bien que certaines actions soient libres, la modification de champs sensibles par un chef de secteur nécessite l'approbation d'un administrateur. Ce processus est accompagné d'un système de notifications intra-application et de courriels automatisés.

II.1.5 Contraintes d'interopérabilité

Enfin, le système devait respecter une contrainte forte d'interopérabilité avec les services de l'État. Les données n'étant pas strictement confidentielles, elles sont rendues accessibles au grand public via le portail gouvernemental MdH. L'application développée intègre donc une fonctionnalité d'export mensuel générant un format de fichier très spécifique, garantissant l'alimentation continue et conforme de ce portail national.

Interlocuteurs, équipe et gestion de projet

Dans le cadre de ce projet, nos interlocuteurs de l'ONaCVG étaient Audrey Paolasini, cheffe du département des achats, Emmanuelle Portugal, archiviste, et Jim Ponty, ancien combattant et chef du secteur de Bordeaux.

Dans la réalisation de ce projet, j'étais accompagné de Matthias en tant que chef de projet, Marine responsable de l'UI/UX, Aurélie en assistance chefferie de projet, rédaction de spécifications et également en développement, Brice en tant que *lead developer*, et Amine et moi-même en tant que développeurs.

Pour chaque fonctionnalité majeure de l'application ont lieu des ateliers entre nos interlocuteurs et Matthias, Marine et Brice. Ces ateliers précisent le cahier des charges initial de l'AO. En découlent des maquettes Figma que Marine réalise et puis des SFD basées sur les maquettes et le cahier des charges. Ensuite, Brice divise la fonctionnalité en tickets et les assigne à Amine ou moi en fonction de nos capacités et disponibilités.

II.2 Migration et regroupement familial

Cette phase du projet a été particulièrement formatrice, marquant ma première immersion dans l'écosystème PHP, le cadriceel Symfony et l'ORM Doctrine. Le défi à relever consistait à consolider une « base mère » et de multiples « bases filles » (fournies sous forme de fichiers CSV). Au sein de ces bases, chaque sépulture

est théoriquement identifiée par un entier unique : la colonne `sdr_num` (numéro de saisie des registres). Cependant, suite à la scission des bases et à l'ajout décentralisé de nouvelles entrées par les chefs de secteur, de nombreuses collisions d'identifiants sont apparues.

Une simple fusion automatisée était inenvisageable en raison de la nature ambiguë de ces collisions. Si deux lignes strictement identiques peuvent être dédoublonnées sans risque, le cas de lignes partageant le même `sdr_num` mais présentant des divergences s'avère complexe. Il peut s'agir d'une véritable collision (deux soldats distincts ayant reçu le même identifiant de manière isolée) ou d'une mise à jour légitime (un même soldat dont les informations ont été enrichies dans une base fille, par exemple avec l'ajout d'un surnom). L'absence de règle mathématique pour trancher ces cas a imposé le développement d'un outil de migration interactif. Cet outil agit comme un POC destinée à détecter les conflits et à déléguer la stratégie de résolution à l'utilisateur.

II.2.1 Choix technologiques et rigueur logicielle

S'agissant d'un projet de réconciliation de données critiques, il était primordial d'établir des fondations techniques solides et de me former aux technologies dont l'application principale fera usage. J'ai donc configuré ce projet sur les dernières normes de l'écosystème avec la pile technologiques classique chez Actimage : PHP 8.5 couplé à Symfony 7.4 et PostgreSQL 18.

Afin de garantir la maintenabilité et la robustesse du code, j'ai intégré un outillage de qualité logicielle exigeant. L'analyse statique du code est assurée par PHPStan poussé à son niveau de vérification maximal (niveau 10), garantissant un typage strict et prévenant les erreurs d'exécution en amont. Le formatage du code est automatisé via PHP CS Fixer, et la fiabilité des algorithmes de résolution de conflits est couverte par des tests unitaires exécutés via PHPUnit. L'interface utilisateur, bien que secondaire pour un POC, utilise le moteur de gabarits Twig couplé au cadriciel Tailwind CSS via le composant Symfony AssetMapper, permettant de se passer d'une lourde chaîne de compilation JavaScript type Node.js/Webpack.

II.2.2 Architecture transactionnelle et asynchrone

L'ingestion de fichiers CSV contenant des centaines de milliers de lignes pose un défi architectural majeur : le traitement synchrone lors d'une requête HTTP entraîne inévitablement un dépassement du temps d'exécution autorisé (timeout) ou une saturation de la mémoire vive.

Pour y répondre, j'ai conçu une architecture asynchrone reposant sur le composant `symfony/messenger` et la bibliothèque d'extraction de données `league/csv`. Le flux de traitement s'articule autour de quatre tables PostgreSQL (TMP, MERE, CONFLICTS et HIST) et se déroule en plusieurs étapes :

Étape 1 : Acquisition et délégation

L'utilisateur téléverse un fichier CSV via la route `/csv-upload`. L'application sauvegarde le fichier sur le disque et délègue immédiatement le travail en publiant un message dans une file d'attente (gérée via le transport Doctrine), libérant ainsi le processus HTTP.

Étape 2 : Traitement asynchrone et optimisation mémoire

Un processus en tâche de fond (le « Worker ») consomme le message. Il utilise des itérateurs générateurs (via `league/csv`) pour lire le fichier ligne par ligne sans jamais charger l'intégralité des données en mémoire. Les lignes sont insérées par lots (batch) dans la table temporaire TMP.

Étape 3 : Répartition automatique

Un algorithme SQL compare ensuite les entrées de TMP avec celles de la base MERE. Les lignes sans conflit d'identifiant y sont intégrées directement. En revanche, les lignes soulevant une collision sur le `sdr_num` sont isolées dans la table CONFLICTS. La table TMP est ensuite purgée de l'entrée fille.

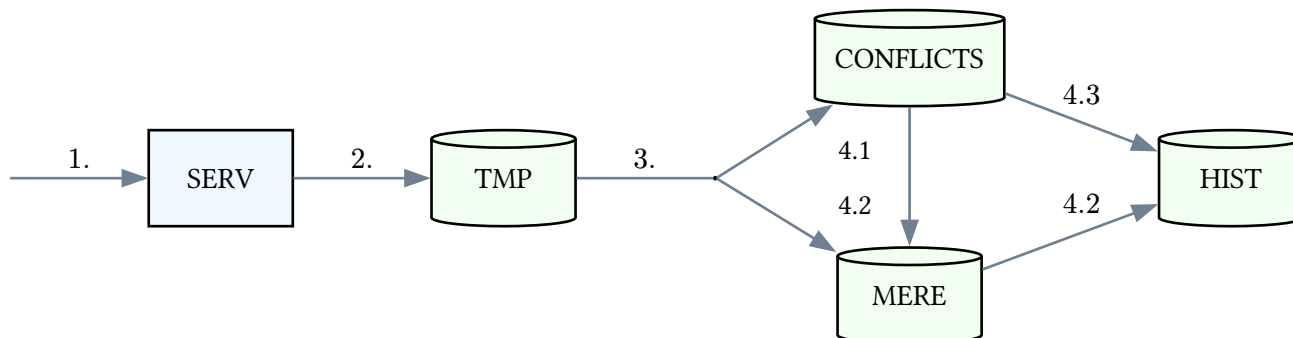


Fig. 3. – Flux de traitement de migration et résolution de conflits

II.2.3 Stratégies de résolution des conflits

L'application propose des interfaces web paginées pour lister et afficher les écarts en exergue (routes / conflicts et /conflicts/{sdrNum}). Face à une collision, l'utilisateur dispose de trois stratégies de résolution (opérées via l'API /resolve/{origDb}/{sdrNum}) :

Action 4.1 : Insertion

Si les deux entrées représentent des soldats différents (vraie collision), l'entrée fille est insérée dans la base MERE (génération d'un nouvel identifiant) et supprimée de CONFLICTS.

Action 4.2 : Écrasement

Si l'entrée fille est une version enrichie et valide de l'entrée mère, l'ancienne entrée mère est archivée dans la table d'historisation HIST. L'entrée fille vient ensuite la remplacer dans MERE via une opération de mise à jour (UPSERT), puis est supprimée de CONFLICTS.

Action 4.3 : Suppression

Si l'entrée fille est jugée erronée ou non pertinente, elle est retirée de la table CONFLICTS et sauvegardée dans HIST afin de conserver une trace de la donnée écartée en cas de besoin futur.

Détails du conflit 1863220

2 conflit(s) à résoudre

← Journal des conflits

NOM DU CHAMP	VALEUR MÈRE	Insérer avec un nouvel ID	Écraser l'entrée mère	Supprimer	Insérer avec un nouvel ID	Écraser l'entrée mère	Supprimer
orig_db	mere-export_b062_sepulture_B.csv	metz-sarrebourg-export_b062_sepulture_B.csv			raon-colmar-export_b062_sepulture_B.csv		
sdr_num	1863220	1863220			1863220		
sdr_num_site1	54467/ROZ	54467/ROZ			54467/ROZ		
sdr_num_conflit	3	3			3		
sdr_nom	BELON	BELON			BELON		
sdr_prenoms	Jean Baptiste Adrien	Jean Baptiste Adrien			Jean Baptiste Adrien		
sdr_grade	Sergent	Sergent			Sergent		
sdr_num_unite	134	134			134		
sdr_unite	R.I.	R.I.			R.I.		
sdr_classe	1910	1909			1909		
sdr_matricule	12R	12R			12R		

Fig. 4. – Capture de la page de résolution de conflits

II.2.4 Ingénierie du déploiement et conteneurisation

Pour assurer l'isolation des processus et garantir la reproductibilité de l'environnement, l'ensemble de ce système a été pensé sous forme de micro-services via Docker Compose. L'environnement orchestre cinq conteneurs interdépendants :

- db : Le moteur de base de données PostgreSQL 18.
- app : Le conteneur principal exécutant l'application Symfony.
- messenger-worker : Un conteneur dédié exclusivement à la consommation des tâches asynchrones en arrière-plan (ingestion des CSV).
- tailwind : Un processus en mode « watch » recompilant à la volée les feuilles de style lors de la modification de l'interface.
- nginx : Le serveur web frontal agissant comme proxy inverse pour exposer l'application sur le port 8080.

II.2.5 Limites du modèle de données plat

Cette phase initiale d'ingestion a mis en évidence les limites physiques du format d'origine. Les exports CSV de la base mère, parfois volumineux (certains générant près de 80 000 conflits et représentant plus de 850 000 entrées au total), étaient traduits littéralement en colonnes PostgreSQL.

Tester l'outil sur de tels volumes a mis en exergue des problématiques de redondance de la donnée et des limites d'indexation. Cela a prouvé la nécessité absolue d'engager, pour le cœur de l'application ECM qui allait suivre, un travail profond de normalisation de la base de données (scission des champs libres en entités fortes avec clés primaires et jointures), sous peine de subir des temps de latence rédhibitoires lors des futures recherches et consultations.

II.3 Modélisation et normalisation de la base ECM

II.3.1 Du modèle plat au modèle relationnel

Une fois la base mère consolidée par l'outil de migration décrit ci-dessus, la donnée restait structurée selon l'héritage historique d'ONaCVG : une unique table plate d'une quarantaine de colonnes, mêlant état civil, données militaires et informations de site sans aucune contrainte d'intégrité référentielle. Un tel modèle, s'il avait été conservé tel quel dans l'application de gestion, aurait rendu impossible toute cohérence à long terme : un même grade ou une même nationalité pouvait être orthographié de multiples façons selon la ligne, sans qu'aucune structure ne permette de les unifier ou de les faire évoluer de manière centralisée.

La refonte du modèle de données a donc consisté à extraire de cette table plate une quinzaine d'entités fortes, jouant le rôle de thésaurus partagés (Grade, Unite, Compagnie, Conflit, SousConflit, CauseDeces, TypeSepulture, Mention, BureauRecrutement, Nationalite, Pays, Departement, Commune, Localisation), reliées à deux entités centrales, Soldat et Site, par des relations ManyToOne gérées via Doctrine ORM. Cette architecture en étoile permet une double garantie : l'intégrité référentielle au niveau base de données (impossible d'associer un soldat à un grade qui n'existe pas) et l'administrabilité de ces référentiels depuis l'interface, sans intervention sur le code.

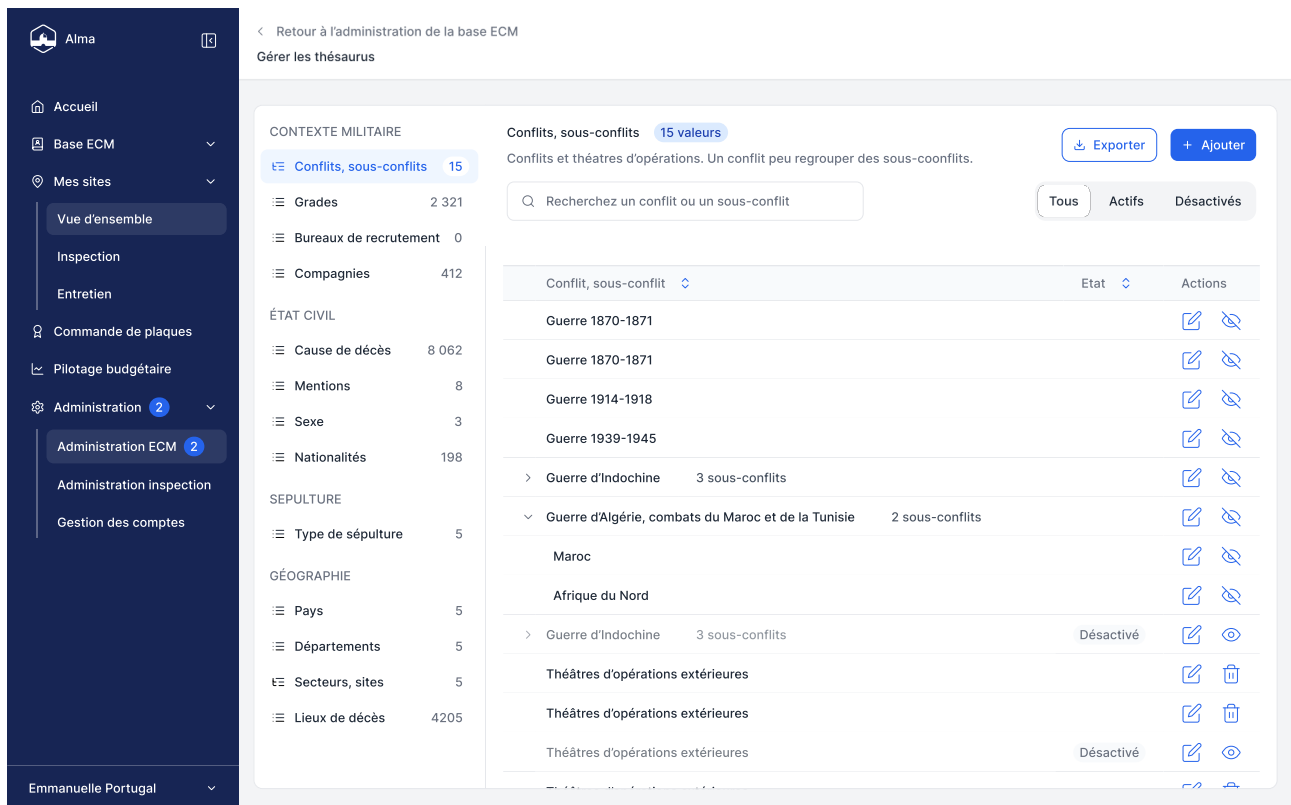


Fig. 5. – Export Figma de l'écran de la page d'administration des thésaurus

II.3.2 Schéma de la base de données

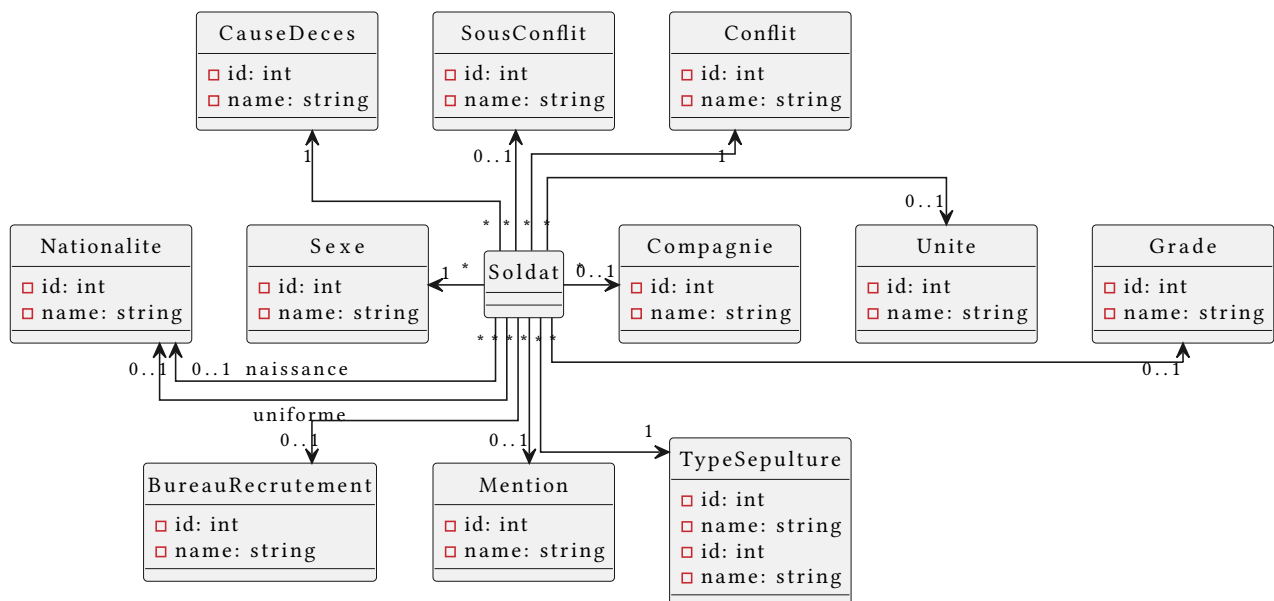


Fig. 6. – Diagramme UML simplifié de l'entité Soldat

L'entité Soldat, avec une quinzaine de clés étrangères, constitue le nœud central du modèle : outre son rattachement à un Site, elle référence indépendamment jusqu'à cinq occurrences de l'entité Localisation (lieu de naissance, de recrutement, de décès, de transcription du décès, de première inhumation), ce qui a permis de conserver la richesse de la donnée d'origine - un soldat pouvant être né, être décédé et avoir été recruté dans trois lieux distincts - sans dupliquer la structure de ces lieux. L'entité Site, de son côté, porte sa propre hiérarchie géographique (Commune, Département, Pays) ainsi qu'une relation inverse vers l'ensemble des soldats qui y sont inhumés, condition nécessaire aux interfaces de recherche par site et aux calculs d'agrégation utilisés dans le module de pilotage.

Donner aux utilisateurs un accès en écriture aux thésaurus est très pratique, puisqu'il leur permet de corriger eux-mêmes leurs erreurs de saisie. Cela impose en retour des validations de données irréprochables : insérer des données propres à la création de la base ne suffit plus, celles insérées par les agents de l'ONaCVG tout au long du cycle de vie de l'application doivent respecter un ensemble de règles précises.

L'exemple principal est celui du triplet Pays, Département et Commune. Tout Département a un Pays, toute Commune a un Pays, et certaines Commune ont un Département¹⁶. La contrainte à faire respecter est donc que pour toute Commune dont le Département est non-NULL, `commune.pays = commune.departement.pays`. Une première approche, simple et efficace, consiste à la vérifier à chaque insertion ou modification via le composant Validator de Symfony, qui évalue un ensemble d'expressions et lève une exception en cas de violation. Efficace, à condition de penser à l'appeler à chaque occasion - une garantie qui ne satisfait pas mon âme de programmeur formé au Coq (aujourd'hui Rocq) et à l'analyse statique.

Un principe que j'aime appliquer est « *Make Illegal States Unrepresentable* »¹⁷. En posant la contrainte directement sur le schéma de la base de données, je délègue sa vérification à PostgreSQL plutôt qu'à PHP - et bien que cette garantie ne soit pas formellement prouvée, je fais infiniment plus confiance à PostgreSQL pour l'appliquer sans exception qu'à un appel de validation qu'on pourrait toujours, par erreur, oublier d'invoquer quelque part dans le code.

PostgreSQL permet nativement de définir des contraintes CHECK sur une colonne ou une table, mais cette fonctionnalité ne fait pas partie du socle SQL commun à tous les moteurs, et Doctrine - qui se veut être une abstraction agnostique du moteur sous-jacent - ne l'expose donc pas. Ma première intuition a été de contourner cette limite avec un attribut PHP personnalisé, `#[Check('...')]`, posé sur une entité Doctrine pour générer automatiquement la contrainte SQL correspondante lors de la création du schéma - une implémentation triviale en apparence. Deux problèmes m'ont fait abandonner cette piste quelques jours plus tard. D'une part, les versions récentes de Doctrine ORM ne permettent plus d'accrocher ce type d'attribut personnalisé au processus de génération de schéma comme prévu initialement. D'autre part, et plus fondamentalement : une contrainte CHECK ne peut évaluer que les colonnes de sa propre table, or `commune.pays_id = commune.departement.pays_id` nécessite de lire une colonne d'une autre table (departement) - ce qu'un CHECK classique ne sait tout simplement pas exprimer, aussi souple soit l'attribut qui le génère.

La solution finale s'est portée vers un outil bien plus commun en SQL, mais employé ici de façon détournée : la contrainte de clé étrangère. Une clé étrangère ne sert habituellement qu'à garantir l'existence d'une ligne référencée dans une autre table ; rien n'empêche cependant de la définir sur plusieurs colonnes à la fois. Il devient alors possible de forcer le couple (`departement_id`, `pays_id`) d'une Commune à exister tel quel dans la table Département, sous la forme (`id`, `pays_id`). Département garantissant déjà que chacune de ses lignes porte un `pays_id` cohérent, cette contrainte composite propage mécaniquement cette cohérence à Commune, sans jamais avoir besoin d'exprimer explicitement une égalité entre deux tables.

Doctrine ne permettant pas de déclarer une clé étrangère composite directement depuis les attributs d'une entité, je l'ajoute au schéma généré via un `DoctrineListener`, écouté sur l'évènement `postGenerateSchema` :

```
<?php
#[AsDoctrineListener(ToolEvents::postGenerateSchema)]
class CommuneConstraintListener
{
    public function postGenerateSchema(GenerateSchemaEventArgs $args): void
    {
        $schema = $args->getSchema();
        $communeTable = $schema->getTable('commune');

        // force commune.pays_id == commune.departement.pays_id
    }
}
```

¹⁶Une Commune en Belgique n'a par exemple pas de Département.

¹⁷Rendre impossibles les états invalides


```

        $communeTable->addForeignKeyConstraint(
            'departement',
            ['departement_id', 'pays_id'],
            ['id', 'pays_id'],
        );
    }
}

```

Lors de la migration suivante, Doctrine compare le schéma de la base avec celui produit par les entités. La création de ce dernier est interceptée par mon `CommuneConstraintListener` qui ajoute insère ce critère dans le schéma. La contrainte apparaît alors naturellement dans la *diff* de la migration, comme n'importe quelle évolution du schéma :

```

ALTER TABLE commune ADD CONSTRAINT FK_E2E2D1EECCF9E01EA6E44244
    FOREIGN KEY (departement_id, pays_id) REFERENCES departement (id, pays_id);

```

De même que la colonne `commune.site_id` porte la contrainte qu'il doit exister, pour toute Commune *c*, un Site *s* tel que `s.id = c.site_id`, ce même mécanisme se généralise à n'importe quel ensemble de colonnes, sans jamais nécessiter la moindre logique de validation applicative.

II.3.3 La chaîne de traitement des thésaurus

Le peuplement initial (puis la mise à jour ponctuelle) de ces référentiels et des entités principales a été industrialisé sous la forme d'une famille de commandes Symfony partageant un socle commun, `AbstractSyncCommand`, paramétré par un type générique (`@template T of Stringable`) correspondant à l'entité ciblée. Chaque commande concrète (`SyncGradeCommand`, `SyncConflitCommand`, `SyncSoldatCommand`, etc., une vingtaine au total) se contente de fournir trois éléments : le ou les fichiers CSV source, le nom de l'entité visée, et une méthode `createEntity()` chargée de transformer un enregistrement brut en instance de l'entité.

La classe abstraite prend alors en charge la mécanique commune : lecture du CSV via `league/csv`, validation de chaque entité créée via le composant `Validator` de Symfony avant persistance, et surtout une gestion fine des erreurs ligne à ligne - une violation de contrainte d'unicité ou un échec de validation n'interrompt pas l'import mais fait l'objet d'un avertissement journalisé et d'un compteur de lignes ignorées, tandis qu'une erreur inattendue interrompt la commande. L'affichage console est lui-même structuré en trois sections indépendantes (erreurs, journal, barre de progression), ce qui permet de suivre en temps réel l'avancement d'un import portant sur plusieurs centaines de milliers de lignes sans noyer les messages d'erreur dans le flux de progression.

```

[INFO] syncing 8060 App\Entity\CauseDeces from cause-deces.csv...

[WARNING] 'mauvais traitements' violates unique constraint, skipping (1)

[WARNING] 'maladie' violates unique constraint, skipping (2)

[WARNING] 'tué par éclats de grenade' violates unique constraint, skipping (3)

[WARNING] 'des suites de blessures' violates unique constraint, skipping (4)

806/8060 [████████████████████████████████████████] 10%

```

Fig. 7. – Capture de la sortie de la commande de synchronisation de tous les thésaurus

II.3.4 La commande d'import principale

SyncSoldatCommand, qui consomme le fichier consolidé par l'outil de migration (mere-dump.csv), illustre le degré d'exigence de cette normalisation. Chaque ligne du CSV plat doit être résolue vers ses entités de référence correspondantes : le conflit renseigné en texte libre (ex. "1914/1918") est d'abord normalisé via une table de correspondance interne avant d'être recherché parmi les entités Conflit existantes, le site est retrouvé par son nom, le type de sépulture par une recherche insensible à la casse, etc. Lorsqu'une correspondance échoue - un site ou un type de sépulture introuvable -, une EntitySyncException dédiée est levée, ce qui fait remonter l'anomalie comme un avertissement exploitable plutôt que comme une erreur silencieuse ou un plantage.

Ce choix (faire échouer explicitement l'import d'une ligne plutôt que de créer une entité incomplète ou incohérente) a constitué un arbitrage central de cette phase : il garantit qu'aucune donnée n'entre dans la base ECM sans que l'ensemble de ses relations obligatoires soit résolu, au prix d'un nombre de lignes rejetées qu'il a fallu analyser et corriger itérativement en amont plutôt qu'en aval sur une base déjà polluée. Il a notamment fallu manuellement corriger les CSV sources (normalisation des nationalités au féminin par exemple), demander au client voire au webmestre de MdH des exports complémentaires pour avoir toutes les données nécessaires ainsi que les liens entre ces dernières.

II.4 Application principale : architecture et réalisation

Une fois la consolidation des données historiques achevée, la phase majeure du projet a consisté à concevoir et développer l'application principale de suivi. Le périmètre fonctionnel, défini par de rigoureuses spécifications, englobe la consultation et la gestion de la base de l'État Civil Militaire (ECM), la saisie d'inspections sur le terrain via tablette, le pilotage budgétaire et la gestion des commandes de plaques.

Bien que l'implémentation de l'ensemble de ces fonctionnalités s'inscrive dans une feuille de route à long terme, mon travail s'est concentré sur la mise en place d'une architecture robuste, d'une chaîne d'intégration continue exigeante et sur le développement des modules centraux.

II.4.1 Architecture technique et paradigme frontal

Pour répondre aux enjeux de maintenabilité et de pérennité du client, la pile technologique s'articule autour des dernières normes de l'écosystème PHP : Symfony 7.4 et PHP 8.5, adossés à une base de données PostgreSQL 18.

L'un des choix architecturaux majeurs a été l'abandon des chaînes de compilation JavaScript lourdes (de type Node.js/Webpack) au profit du composant Symfony AssetMapper. Ce paradigme moderne permet de gérer les dépendances client (JavaScript et CSS) directement via PHP. L'interface utilisateur est ainsi propulsée par le moteur de gabarits Twig, stylisée dynamiquement via le cadriciel Tailwind CSS (compilé nativement), et rendue interactive grâce à Stimulus (Symfony UX). Cette approche réduit drastiquement la complexité de l'infrastructure de déploiement tout en garantissant des performances optimales côté client.

II.4.2 Modélisation des droits et rôles

Les spécifications fonctionnelles définissent huit rôles distincts, chacun associé à une étendue de visibilité nationale ou limitée à un ou plusieurs secteurs géographiques (Fig. 8). Ce référentiel, issu d'une matrice rôles/droits établie en amont avec le client, a été traduit côté code par un enum PHP dédié (App\Security\Role), dont chaque cas porte son libellé métier via une méthode label() - un choix qui évite la dispersion de chaînes de caractères représentant les rôles à travers l'application et centralise leur nommage.

Rôle	Étendue	Accès principal
Superadmin	National	Toutes données, administration complète
Administrateur ECM	National	Lecture/écriture base ECM, administration, commande de plaques
Administrateur Inspection	National / Secteur	Lecture/écriture inspections, administration des référentiels de secteur
Chef de secteur	Secteur	Lecture base ECM, écriture limitée à son secteur, commande de plaques
Consultant ECM	National	Lecture seule base ECM
Consultant Inspection	Secteur	Lecture seule sites/inspections de son secteur
Consultant Budgétaire	National	Lecture seule pilotage budgétaire
Consultant	National	Lecture seule transverse (ECM, sites, pilotage)

Fig. 8. – Rôles applicatifs et étendue associée

Côté implémentation, la hiérarchie entre ces rôles est déclarée dans la configuration de sécurité de Symfony (`security.yaml`), où `ROLE_SUPERADMIN` hérite automatiquement de l'ensemble des autres rôles évitant ainsi de dupliquer les vérifications d'accès pour l'administrateur global. Le contrôle d'accès aux actions sensibles (édition d'un soldat ou d'un site) est réalisé au niveau des contrôleurs via l'attribut `#[IsGranted('ROLE_ADMIN_ECM')]` de Symfony Security, appliqué directement sur les méthodes concernées.

Le second axe du cloisonnement des droits la restriction géographique par secteur, qui borne par exemple l'écriture d'un chef de secteur aux seuls sites de son secteur - s'appuie sur une entité `Secteur` nouvellement introduite (liaison `ManyToOne` depuis `Site`), actuellement développée sur une branche dédiée et pas encore fusionnée à main au moment de la rédaction de ce rapport. Cette entité pose les fondations de données nécessaires à une future logique d'autorisation par `voter` Symfony, qui viendra comparer le secteur de rattachement de l'utilisateur à celui de la ressource consultée - mécanisme non encore implémenté à ce stade du projet.

II.4.3 Rigueur logicielle et Intégration Continue

Afin de garantir que le code produit par l'équipe réponde aux standards de qualité de l'ingénierie logicielle, j'ai participé à la mise en place d'une chaîne d'intégration et de déploiement continu (*pipeline* Jenkins) particulièrement stricte. Chaque demande de fusion (*merge request*) doit valider quatre familles d'étapes bloquantes avant d'être intégrée à la branche principale : construction et linters (container Symfony, YAML, Twig, mapping Doctrine), tests, analyse statique (PHPStan, PHP CS Fixer, Twig CS Fixer, Biome, Rector), puis sécurité (audit Composer, Gitleaks) les trois dernières familles s'exécutant en parallèle pour limiter le temps de retour.

Crochets Git

Afin de raccourcir la boucle de rétroaction et d'éviter qu'une violation de style ou la compromission d'un secret ne soient découvertes tardivement lors de la demande de fusion, des *hooks* Git locaux (`.github/hooks/pre-commit` et `.github/hooks/pre-push`) ont été mis en place. Ils s'activent automatiquement via `make hooks` lors de l'initialisation du projet. Le hook de pré-commit exécute localement un sous-ensemble rapide des vérifications de l'intégration continue sur les fichiers modifiés (formatage PHP, Twig et JavaScript, ainsi qu'un scan Gitleaks).

Cette approche répartit intelligemment la charge de vérification : le poste du développeur offre un retour quasi-instantané sur les erreurs courantes, tandis que le pipeline CI garantit une validation exhaustive et incontournable avant intégration, le tout sans dupliquer la configuration des outils.

Toutefois, ce mécanisme présente certaines limites d'usage. Notamment, lorsqu'un développeur modifie plusieurs fichiers mais souhaite n'en valider (*commit*) qu'une partie, les vérifications de qualité s'appliquent parfois à l'ensemble du répertoire de travail, sans distinguer les fichiers indexés (*staged*) de ceux qui ne le

sont pas. Dans cette situation spécifique, le développeur conserve la flexibilité de contourner ponctuellement les crochets à l'aide de l'option `--no-verify` [4].

Analyse statique et typage strict

Le code est analysé par PHPStan, configuré à son niveau d'exigence maximal (Niveau 9), le plus strict proposé par l'outil. Pour absorber la dette technique déjà présente sans bloquer immédiatement toute la base de code, une ligne de base (baseline) archive l'ensemble des erreurs détectées à un instant donné dans un fichier dédié (`phpstan-baseline.neon`) : ces erreurs existantes sont ignorées lors des analyses suivantes, mais toute nouvelle erreur, elle, fait immédiatement échouer la vérification. Cette liste ne peut donc que diminuer au fil du temps - régénérer la ligne de base après avoir corrigé une erreur retire mécaniquement son entrée -, ce qui permet d'imposer une rigueur maximale sur tout code nouvellement écrit sans exiger une remise à niveau complète et immédiate de l'existant.

Formatage et refactorisation

La syntaxe du projet est uniformisée par PHP CS Fixer côté PHP et par Twig CS Fixer côté gabarits Twig. Ces deux outils ne sont, à proprement parler, pas des formateurs : ce sont avant tout des moteurs de vérification, qui inspectent le code à la recherche d'infractions à un ensemble de règles configurées. C'est uniquement parce que chaque règle est associée à sa propre correction automatique que ces outils peuvent, en pratique, être invoqués comme de simples formateurs.

Twig CS Fixer illustre bien cette distinction : parce qu'il opère au niveau du *lexer* et du *Node* Twig plutôt que sur le fichier brut, il ne touche jamais au HTML dans lequel la syntaxe Twig est entremêlée - il se limite à vérifier et corriger l'espacement autour des opérateurs, des délimiteurs de bloc et des arguments nommés, ainsi que les conventions de nommage des fichiers et répertoires, sans jamais reformater la structure du document. Le projet applique le standard Symfony fourni par l'outil, qui étend le standard Twig de base (espacement des opérateurs, guillemets simples, virgules finales) avec des règles supplémentaires propres aux conventions Symfony, telles que le nommage en `snake_case` des fichiers et répertoires de gabarits.

Le code JavaScript est quant à lui audité par Biome [26], qui joue ce même double rôle de linter et de formateur sur le périmètre des assets applicatifs. Enfin, l'outil Rector est intégré en mode vérification (`dry-run`, sans application automatique des changements) pour suggérer des refactorisations plus profondes que le simple style - suppression de code mort, simplifications sémantiquement équivalentes, ou ajout de déclarations de type manquantes -, dont chaque suggestion reste soumise à une relecture humaine avant d'être appliquée.

Sécurité

Outre l'audit des dépendances Composer, la CI intègre Gitleaks, un outil scannant l'historique des modifications (via des expressions régulières) pour empêcher la fuite de secrets ou de clés d'API dans le code source.

Stratégie de tests

La suite de tests, exécutée via PHPUnit et pilotée par un fichier `phpunit.dist.xml` dédié, s'organise selon trois niveaux de granularité croissante :

- des **tests unitaires purs** (`PHPUnit\Framework\TestCase`), qui isolent la logique métier de toute dépendance au framework par exemple la sérialisation des critères d'une recherche sauvegardée (`SavedSearchTest`), où les entités liées sont simulées via des *stubs* plutôt qu'instanciées en base ;
- des **tests d'intégration** (`Symfony\Bundle\FrameworkBundle\Test\KernelTestCase`), qui démarrent un noyau applicatif réduit pour valider des composants dépendant de Doctrine, tels que les requêtes de recherche multicritère du `SoldatRepository`, souvent pilotées par des jeux de données paramétrés (`#[DataProvider]`) ;
- des **tests fonctionnels** (`WebTestCase`), qui simulent un navigateur HTTP complet (`KernelBrowser`) pour valider des parcours utilisateurs de bout en bout soumission d'un formulaire de recherche, tri des résultats, disponibilité générale de l'application.

Ces deux derniers niveaux s'appuient sur un système de fixtures Doctrine organisées par dépendances explicites (`DependentFixtureInterface`) et groupées (`FixtureGroupInterface`) : le groupe `test`, chargé uniquement pour les tests, garantit un jeu de données minimal et reproductible sans intervenir sur les données de développement. Chaque test de base de données s'exécute par ailleurs au sein d'une transaction automatiquement annulée grâce au paquet `dama/doctrine-test-bundle`, ce qui garantit à la fois l'isolation entre tests (aucun effet de bord d'un test à l'autre) et des performances d'exécution nettement supérieures à un rechargement complet du schéma entre chaque cas.

Des tests fonctionnels d'interface sont également prévus mais non encore implémentés. Selon l'habitude chez Actimage, c'est le cadriciel de test Playwright [27], qui sera utilisé pour cette partie.

II.4.4 Implémentation des règles métiers et sécurité

Le cahier des charges impose une gestion fine des habilitations, réparties selon plusieurs rôles : Superadmin, Administrateur ECM, Administrateur Inspection, Chef de secteur et différents profils de consultants. Les droits d'accès ont été modélisés via le composant Security de Symfony (Voters et hiérarchie des rôles), la hiérarchisation des rôles étant en place dès le socle applicatif, tandis que le cloisonnement fin par étendue géographique (National vs Secteur), en cours de développement à la fin du stage, s'appuie sur l'entité Secteur nouvellement introduite dans le modèle de données

Le cœur du système repose sur la Base ECM, exigeant des interfaces de recherche multicritères complexes (par individu ou par site). La modélisation a nécessité de relier les entités `Soldat` et `Site` à un riche système de thésaurus (grades, conflits, unités, causes de décès) gérable dynamiquement par les administrateurs.

II.4.5 Défis techniques des modules fonctionnels

Bien que l'application comporte de nombreux modules, certains ont représenté des défis techniques et ergonomiques particulièrement intéressants :

Consultation et recherche multicritère

Le module de consultation constitue le point d'entrée quotidien des utilisateurs vers la base ECM. Il repose sur un objet de requête dédié (`SoldatSearchData`), une simple classe de données publiques portant l'ensemble des critères de filtrage disponibles nom, prénoms, dates de naissance et de décès, département de naissance/décès, conflit, site d'inhumation, présence de sépulture, caractère perpétuel de la sépulture - ainsi que les paramètres de pagination et de tri. Ce découpage isole complètement l'expression d'une recherche de son exécution : le contrôleur se contente d'hydrater cet objet depuis les paramètres de la requête HTTP, avant de le transmettre au dépôt Doctrine (`SoldatRepository::search()`).

Le dépôt construit alors dynamiquement une requête Doctrine Query Language¹⁸ (DQL) via le `QueryBuilder` de Doctrine, n'ajoutant une clause `WHERE` ou une jointure que pour les critères effectivement renseignés, évitant ainsi de générer un unique DQL statique avec de multiples conditions optionnelles peu lisibles et des jointures coûteuses et inutiles, au profit d'une construction incrémentale et testable indépendamment critère par critère (cf. `SoldatRepositoryTest` et son usage de `#[DataProvider]`). Certains critères, comme la recherche textuelle sur le nom, exposent également un mode de correspondance paramétrable (`TextSearchType` : contient, commence par, exact), traduit en clause SQL `LIKE` ou `=` selon le cas.

Pour répondre au besoin d'un usage récurrent de certaines combinaisons de filtres, une fonctionnalité de recherche sauvegardée (`SavedSearch`) a également été développée : elle sérialise l'état complet d'un `SoldatSearchData` sous forme de chaîne de requête, réutilisable pour reconstituer une recherche identique en un clic depuis le tableau de bord de l'utilisateur.

¹⁸Le langage de requête orienté objet utilisé par Doctrine plutôt que d'écrire du SQL cru

Alma

Accueil

Base ECM

Rechercher

Mes sauvegardes

Mes exports

Mes sites

Commande de plaques

Pilotage budgétaire

Administration

Emmanuelle Portugal

Rechercher dans la base de l'État civil militaire

Par individu

Par site

Nom

Ex : Dupont

Prénoms

Ex : Louis Paul Jean

Date de naissance

JJ/MM/AAAA

Département de naissance

Sélectionnez un département

Date de décès

JJ/MM/AAAA, MM/AAAA ou AAAA

Département de décès

Ain

Lieu de décès

Sélectionnez un lieu

Conflit

Sélectionnez un conflit

Département d'inhumation actuelle

Sélectionnez un département

Site d'inhumation actuelle

Sélectionnez un site

Soldat inconnu

Inclure Exclure Uniquement

Présence de la sépulture

Sépulture existante

Sépulture perpétuelle

Tous

Tout effacer

Filtres avancés

Rechercher

Pays de naissance : Allemagne

Localisation de la tombe : Carré : 18 / Rang : 15 / Numéro : 12

Matricule de recrutement : 2154

45 résultats trouvés

Exporter

Nom	Prénoms	Date naissance	Dép. naissance	Date décès	Dép. décès	Lieu décès	Conflit	Dép.
GRIMAUD	Auguste	01/12/1902	Ain (01)	05/11/1918	Meuse (55)	Verdun	1914-1918	Meu
DUPONT	Pierre	15/05/1899	Paris (75)	12/04/1917	Somme (80)	Albert	1914-1918	Son
MOREAU	Sophie	29/11/1875	Marseille (13)	17/05/1918	Meuse (55)	Verdun	1914-1918	Meu
BENOIT	Louis	05/09/1892	Lille (59)	30/10/1915	Nord (59)	Lille	1914-1918	Nor
GARNIER	Émile	12/07/1880	Strasbourg (67)	09/08/1917	Aisne (02)	Reims	1914-1918	Aisr
CHEVALIER	François	18/04/1893	Nice (06)	11/04/1918	Oise (60)	Compiègne	1914-1918	Oisr
ROUSSEL	Alice	30/01/1891	Toulouse (31)	14/06/1916	Somme (80)	Péronne	1914-1918	Son
LACROIX	Hugues	24/09/1895	Nantes (44)	20/07/1917	Marne (51)	Châlons-en...	1914-1918	Mar
NOEL	Julien	01/12/1894	Bordeaux (33)	25/08/1918	Meuse (55)	Verdun	1914-1918	Meu
LEROY	Cécile	17/02/1890	Avignon (84)	07/11/1916	Aisne (02)	Laon	1914-1918	Aisr
DUMONT	Philippe	09/03/1885	Toulon (83)	19/04/1917	Pas-de-Cal...	Arras	1914-1918	Pas
BOURGEOIS	Emilie	21/08/1892	Lille (59)	12/05/1916	Somme (80)	Montdidier	1914-1918	Son
BLANCHARD	Gilbert	13/06/1889	Nancy (54)	22/07/1918	Meuse (55)	Verdun	1914-1918	Meu
FONTENEAU	Juliette	05/10/1893	Dijon (21)	15/09/1916	Nord (59)	Douai	1914-1918	Nor
LEGENDRE	Bastien	28/12/1891	Rouen (76)	26/10/1917	Aisne (02)	Château-T...	1914-1918	Aisr
LAMARRE	Louise	14/11/1887	Limoges (87)	03/11/1916	Marne (51)	Épernay	1914-1918	Mar

25 50 100 lignes par page

« 1 sur 3 »

Fig. 9. – Export Figma de l'écran de recherche par individu avec résultats

Gestion des inspections en mobilité

L'application prévoit un module d'inspection destiné à être utilisé par les chefs de secteur sur tablette, directement sur les sites. L'interface a dû être pensée pour le format tactile (boutons larges, listes déroulantes optimisées). D'un point de vue technique, ce module intègre la capture de photographies via l'appareil de la tablette et l'application d'actions par lots (ex: dupliquer l'état de dégradation d'une stèle sur une plage de tombes définie par leurs numéros de rangs et de carrés).

Les sites étant souvent situés dans des endroits reculés, en particulier pour les carrés militaires que l'on peut trouver dans des cimetières de villages, voire de hameaux, le chef de secteur les inspectant n'aura pas toujours accès à internet. La solution envisagée est le téléchargement préalable des données nécessaires dans le cache du navigateur, permettant ainsi un usage totalement hors-ligne. Une fois l'inspection terminée et la tablette placée dans un lieu avec accès internet, le formulaire d'inspection peut être soumis et enregistré sur la base de données de l'application.

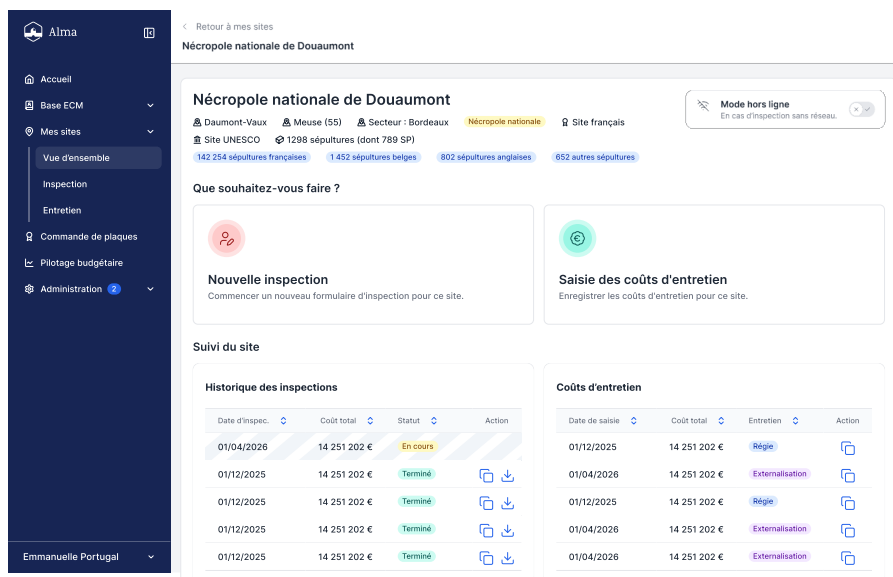


Fig. 10. – Export Figma de l'écran de gestion d'un site (vue du chef de secteur)

Flux de validation (Workflow)

Pour protéger l'intégrité des données historiques (sépultures perpétuelles, mentions « Mort pour la France »), les spécifications fonctionnelles définissent un flux de validation structuré : toute demande de suppression de fiche, de création de site ou de modification d'un champ réglementaire soumise par un chef de secteur ne s'applique pas directement, mais transite par un état « en attente » jusqu'à l'arbitrage d'un Administrateur ECM, avec obligation de motiver un refus.



Fig. 11. – Export Figma la modale de validation des modifications

Interopérabilité et Exports

Le système devait respecter une contrainte forte d'interopérabilité avec les services de l'État. Les données n'étant pas strictement confidentielles, elles sont rendues accessibles au grand public via le portail gouvernemental MdH. Les spécifications fonctionnelles définissent à ce titre une page d'export dédiée, proposant

deux modes : un export total de la base ECM, et un export partiel piloté par un assistant en trois étapes, permettant de restreindre l'export soit aux dernières mises à jour depuis une date donnée, soit à une sélection explicite de sites. Chaque export généré transite par un état intermédiaire (« en cours de génération », visible depuis la page dédiée) avant sa mise à disposition au téléchargement, avec gestion explicite des échecs et possibilité de relance.

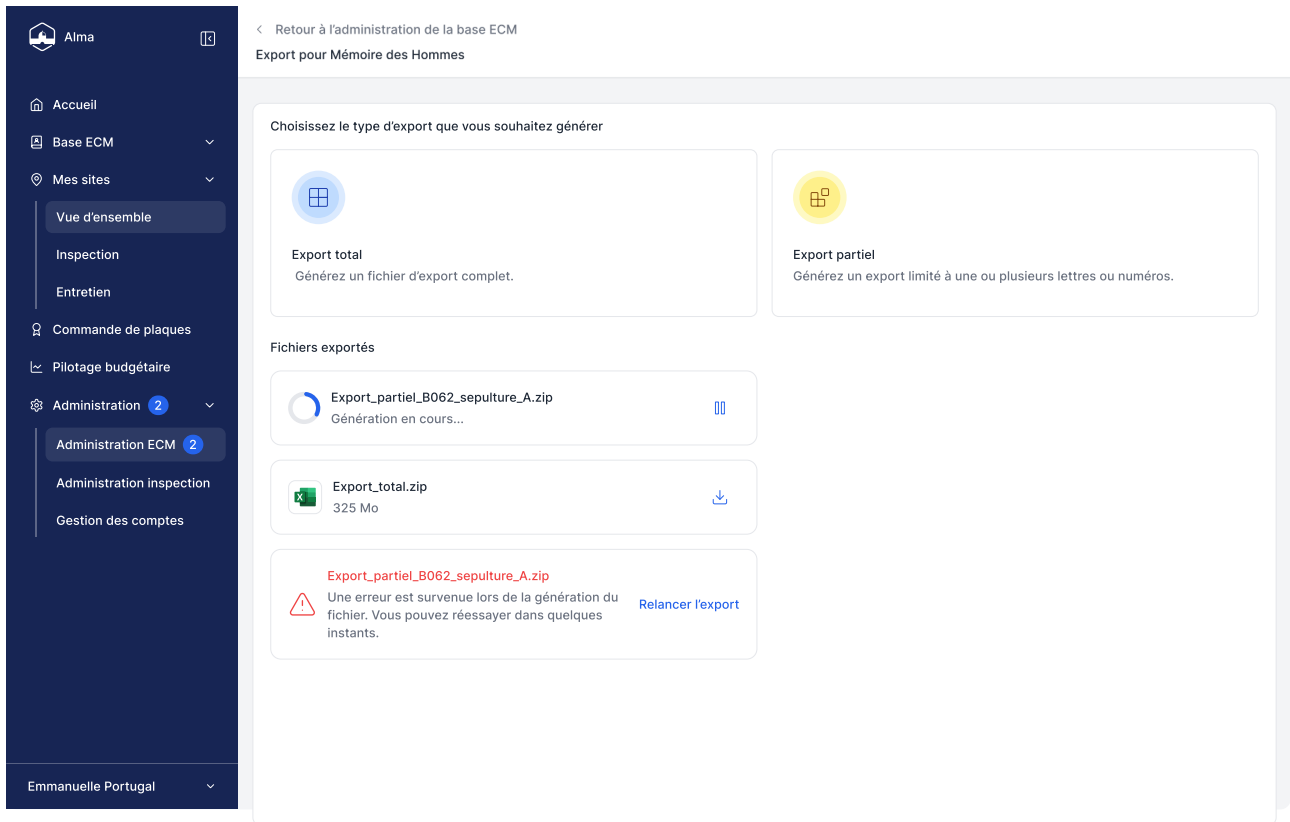


Fig. 12. – Export Figma de l'écran de l'écran d'export vers MdH

À l'instar du flux de validation, cette fonctionnalité d'export n'était pas encore implémentée dans le périmètre développé pendant mon stage : seule sa maquette visuelle existe à ce stade, matérialisée dans le *design system* interne de l'application par des composants de notification réutilisables (succès, information, erreur) illustrant les différents états d'un export (génération en cours, fichier disponible, échec - sans que la génération du fichier plat destiné à MdH n'ait elle-même été codée).

II.5 Bilan technique et perspectives

Le projet ONaCVG est de loin celui sur lequel j'ai passé le plus de temps durant ce stage. Comme indiqué plus haut, son développement a débuté en même temps que mon arrivée. Le choix d'Actimage pour mon PFE tenait notamment à la taille humaine de l'entreprise, des équipes et des projets entrepris. Ma prédiction s'est réalisée : si ce projet me tient autant à cœur, c'est parce qu'il est assez complexe pour être profondément intéressant, sans que ses dimensions ne dépassent ma compréhension. Je peux suivre la quasi-totalité des lignes de code et des processus du projet, bien qu'il s'étende sur près de 400 fichiers.

Le point technique qui m'a le plus découragé a été la mise en place de la logique côté client, dans un contexte sans outil puissant d'analyse statique l'application se passant volontairement de TypeScript au profit de JavaScript natif. Cette absence de garde-fou à la compilation a fini par orienter mon choix vers Stimulus, le cadriciel recommandé par Symfony UX : en imposant une structure déclarative (contrôleurs, cibles, valeurs) directement dans le HTML plutôt que de la logique JavaScript libre, il compense en partie l'absence de typage en réduisant fortement la surface d'erreur possible, tout en restant cohérent avec le choix architectural d'AssetMapper.

À l'inverse, ce que je suis le plus fier d'avoir mis en place côté PHP, c'est l'effort systématique consistant à déplacer un maximum de logique métier dans les types et les structures du code plutôt que dans des vérifications à l'exécution les enums stricts (`PresenceSepulture`, `TextSearchType`, `Role`), le typage strict imposé par PHPStan niveau 9, ou encore l'usage des traits Symfony pour partager du comportement (`LoggerTrait`, `AlertControllerTrait`) sans passer par une hiérarchie d'héritage rigide. Le typage statique, quand il est poussé sérieusement, est un outil remarquable : il permet de prouver, au sens quasi mathématique du terme, qu'une certaine classe de bugs ne pourra tout simplement pas se produire à l'exécution. Les traits, de leur côté, permettent de réutiliser du code transversal sans imposer la contrainte d'un seul parent, libérant ainsi une charge mentale de conception que l'héritage classique aurait alourdi.

Avec le recul, s'il y a un choix que je referais différemment, ce serait d'imposer des standards de code plus exigeants que ceux couverts par Rector et PHP CS Fixer, qui restent essentiellement syntaxiques. Je verrais volontiers l'ajout, à la chaîne d'intégration continue existante, d'un bloc de revue automatisée s'appuyant sur un modèle de langage (à la manière d'outils comme CodeRabbit), capable de commenter directement une demande de fusion sur des questions de style et de bonnes pratiques que les outils actuels ne couvrent pas : nommage, cohérence des patterns entre modules, lisibilité - et d'accélérer ainsi l'uniformisation du projet sans alourdir la charge de relecture humaine du *lead developer*.

Ce projet a par ailleurs été autant stimulant humainement que techniquement. Les échanges avec Jim, chef de secteur à Bordeaux et lui-même ancien combattant, ont concrètement ancré mon travail : l'entendre exposer les problématiques pratiques de son quotidien d'inspecteur de site, l'éloignement de certains carrés militaires, la difficulté de maintenir une base de données à jour sur le terrain a donné un sens tangible à des choix qui, sur le papier, n'étaient que des arbitrages techniques (mode hors-ligne, ergonomie tactile du formulaire d'inspection).

Sur le plan de ma formation d'ingénieur, ce stage a surtout été une leçon de pragmatisme technique. Venant d'une formation où des langages fortement typés comme OCaml ou Java sont mis en avant pour leur rigueur, j'ai appris qu'un langage moins strict par nature PHP, historiquement permissif peut être « repris en main » via un outillage de qualité suffisamment exigeant (PHPStan au niveau maximal, validation Symfony, tests) pour retrouver une bonne part des garanties recherchées, tout en conservant une vélocité de développement que des langages plus rigides auraient difficilement permise sur un projet de cette taille et avec ces délais.

III PIAWEB : Une histoire de DevOps

L'application PIAWEB¹⁹ dont Actimage réalise les développements et dirige les déploiements sur les serveurs clients est un projet de répertoire pour suivre les différentes actions du plan d'investissement France 2030 qui relèvent spécifiquement du Ministère de l'Enseignement Supérieur et de la Recherche (MESR).

Ce projet est actuellement en phase de TMA, peu de développements sont réalisés, majoritairement des corrections de bogues ou des évolutions mineures. La pile technologique repose sur du Spring et du Angular, une solution légèrement plus élaborée que celle proposée pour l'ONaCVG puisqu'elle requiert deux conte-neurs serveur distincts pour le web frontal et dorsal.

III.1 Anatomie d'une infrastructure industrialisée

III.1.1 Architecture de l'environnement DevOps et de l'infrastructure

L'hébergement et le déploiement de PIAWEB s'appuient sur une infrastructure robuste et standardisée, représentative des bonnes pratiques du pôle Digital d'Actimage. L'ensemble des environnements est conte-neurisé à l'aide de Docker, ce qui garantit une isolation parfaite des processus et une reproductibilité des déploiements.

L'architecture matérielle du serveur alloué au projet se distingue par la présence d'un disque supplémentaire de 100 Go, géré et partitionné dynamiquement via LVM (Logical Volume Manager). Cet espace est stratégiquement découpé :

- Un volume LVM de 70 Go est monté sur le répertoire `/srv/`. Ce répertoire centralise l'installation de Docker, les différentes instances déployées du projet, ainsi que les exécuteurs (runners) GitLab. Par défaut, le répertoire d'installation de Docker a d'ailleurs été reconfiguré pour pointer vers `/srv/docker` afin d'exploiter cet espace de stockage étendu.
- Un second volume LVM de 30 Go est monté sur `/srv/registry/` et est exclusivement dédié aux besoins du registre d'images (Harbor), permettant de stocker les images Docker générées.

L'accès aux différents services web est orchestré par un serveur mandataire inverse (reverse-proxy) Nginx. Les configurations d'accès sont gérées classiquement via les répertoires `/etc/nginx/sites-available` et `sites-enabled`. Afin de protéger les environnements hors production, une restriction d'accès de type `auth_basic` est implémentée. Différents fichiers d'identifiants (`.htpasswd_actimage`, `.htpasswd_client`, `.htpasswd_all`) sont utilisés pour cloisonner l'accès selon qu'il s'agisse de l'environnement d'intégration (réservé à Actimage) ou de recette (ouvert au client).

La pile applicative elle-même est divisée en trois conteneurs principaux :

Backend

Développé en Java 17 avec Springboot 2.7, ce module expose les services de l'application et se connecte à la base de données.

Frontend

Les interfaces utilisateur en Angular, dont les composants transpilés sont servis par un serveur Nginx interne et autonome.

Flyway

Un conteneur éphémère dédié exclusivement à la gestion et à l'exécution des scripts de migration SQL pour faire évoluer la structure de la base de données PostgreSQL à chaque changement des entités qui cause un changement de schéma de table. Ce conteneur se lance lorsque l'application est mise en ligne et s'arrête dès que les migrations sont effectuées.

¹⁹Contraction de Programme d'Investissements d'Avenir (PIA) et de Web.

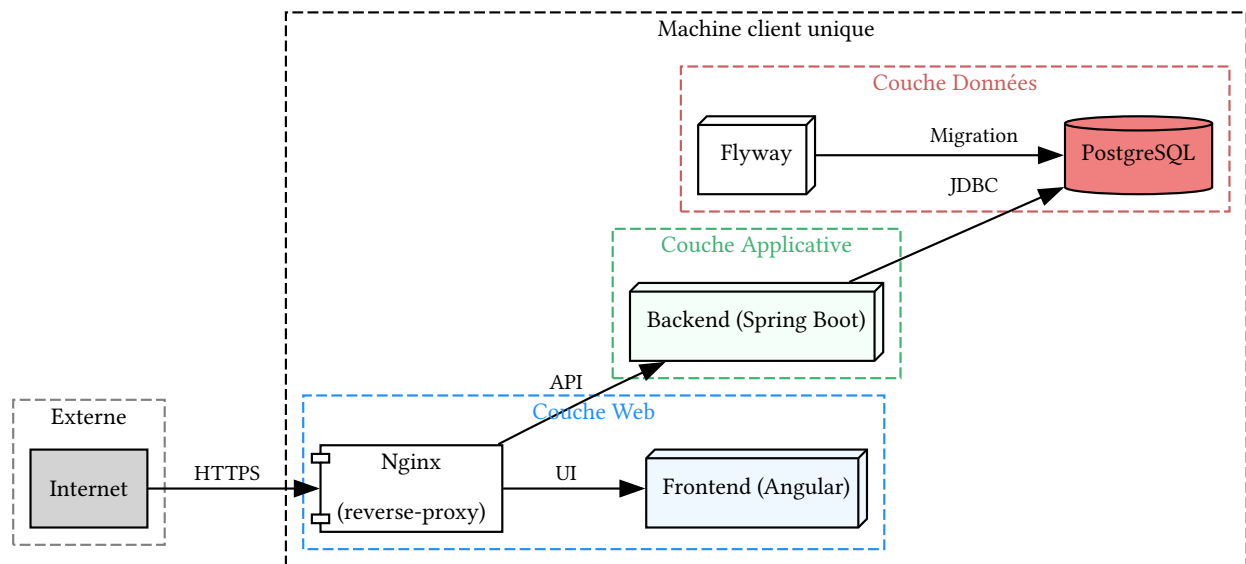


Fig. 13. – Architecture physique et logique de l'infrastructure PIAWEB

III.1.2 Intégration Continue et Déploiement Continu (CI/CD)

Afin de fluidifier le cycle de développement et de garantir la qualité du code, le projet s'appuie sur la plateforme GitLab pour son intégration continue. L'organisation du code est modulaire : un macro-projet centralise la configuration CI/CD, tandis que les différents composants (frontend, backend, base de données, infrastructure Docker) sont gérés sous forme de sous-modules Git.

L'exécution des tâches de la CI/CD est confiée à des GitLab Runners installés manuellement sur le serveur d'intégration. L'environnement s'appuie sur deux types d'exécuteurs :

- Un **runner Docker** (runner-gitlab-piaweb-int) : utilisé pour exécuter les tâches dans des conteneurs isolés, garantissant des environnements de construction propres et jetables.
- Un **runner Shell** (runner-gitlab-piaweb-shell) : configuré pour s'exécuter directement sur la machine hôte via un utilisateur système dédié (gitlab-runner). Bien que son usage soit généralement déconseillé car il peut laisser des fichiers résiduels, il est parfois nécessaire pour interagir directement avec l'infrastructure du serveur d'intégration.

Les chaînes de traitement sont configurés pour se déclencher selon des événements précis : lors de la soumission de code sur une branche spécifique, lors de la création d'une étiquette (tag), ou lors d'une publication (release).

III.1.3 Gestion des environnements et stratégie de branche

Le dépôt principal de PIAWEB s'organise en un projet racine, piaweb, qui référence quatre sous-modules Git : backend, frontend, database et docker. Cette centralisation permet de cloner l'ensemble du projet en une seule opération plutôt que de devoir cloner puis synchroniser individuellement chaque composant - un confort non négligeable pour l'initialisation d'un environnement de développement ou d'intégration.

Le cycle de vie du code de PIAWEB est rythmé par le passage à travers différents environnements, chacun répondant à un besoin spécifique et associé à des stratégies de branches Git rigoureuses. Sur les dépôts backend, frontend et database, cette rigueur se traduit par une branche dédiée à chaque environnement - dev, int, rec, et une dernière, prod, mutualisée entre pré-production et production puisque ces deux environnements partagent déjà les mêmes images Docker (voir ci-après). Une modification suit alors un flux de promotion classique : une évolution est développée et revue sur dev, fusionnée vers int pour y être testée en continu, puis vers rec pour validation client, et enfin vers prod pour la mise en production. Chaque fusion représente ainsi une étape de validation franchie, et l'historique Git de chaque branche reflète fidèlement l'état du code effectivement déployé sur l'environnement correspondant.

De dev vers int (Intégration)

Les environnements de développement local et d'intégration sont techniquement identiques. Ils exploitent les outils de rechargement à chaud (comme Springboot DevTool). À ce stade, le code source n'est pas « figé » dans les images Docker, mais monté via des volumes, ce qui évite de devoir reconstruire les images à chaque modification et accélère considérablement le cycle de développement. La mise à jour de l'environnement d'intégration se fait via de simples commandes `git pull` et `git submodule update`, suivies d'une compilation Maven (`mvn clean install`), tout ceci facilité par l'usage de clés de déploiement (Deploy Keys) configurées sur la machine virtuelle.

De int vers rec (Recette)

C'est lors du passage en recette que le paradigme change radicalement. Le code applicatif est désormais compilé et intégré en dur au sein même des images Docker. Ce figeage garantit que l'image testée par le client sera strictement identique à celle qui sera mise en production.

De rec vers prep (Pré-production)

Il s'agit d'environnements différents, mais qui exploitent les mêmes images Docker. Cette étape permet de valider le comportement de la version packagée dans une infrastructure imitant la production.

De prep vers prod (Production)

L'environnement et les images Docker restent les mêmes que lors de l'étape de pré-production. L'enjeu ici n'est plus technique mais critique : appliquer la mise à jour sans provoquer d'interruption de service ou d'anomalie sur le système en exploitation.

III.1.4 Un anti-pattern relevé : la stratégie de branches du dépôt docker

Ce même découpage en une branche par environnement (dev, int, rec, prep/prod) est également appliqué au sous-module docker, qui héberge les fichiers `Dockerfile` et `docker-compose.yml` de chaque environnement. Ce choix me semble constituer une utilisation abusive des branches Git, pour une raison simple : une branche est, par nature, un outil destiné à isoler un travail appelé à converger - une fonctionnalité en cours de développement, une correction en attente de revue - avant d'être fusionné dans une autre branche. Or les méthodes de déploiement diffèrent fondamentalement d'un environnement à l'autre : en dev et int, le code source est monté en volume dans les conteneurs pour bénéficier du rechargement à chaud, tandis qu'à partir de rec, il est figé et compilé en dur dans l'image. Les fichiers de configuration Docker de ces environnements ne sont donc pas des variations d'un même travail en cours, mais des configurations durablement distinctes, qui n'ont jamais vocation à se fusionner les unes dans les autres.

Cette absence de convergence prévue pose un problème concret dès qu'un changement doit s'appliquer à plusieurs environnements à la fois - la mise à jour d'une version de base d'image, ou la correction d'un nom de service mal orthographié, par exemple. N'ayant pas de branche commune vers laquelle ces branches convergent, un tel changement ne peut être propagé qu'en le répliquant manuellement sur chacune d'entre elles, typiquement par *cherry-pick* - une opération répétitive, sujette à l'oubli d'une branche, et qui ne laisse aucune trace du fait que ces N commits représentent en réalité une seule et même intention de changement.

Le sous-module docker illustre pourtant un cas d'école pour lequel Docker Compose propose nativement une meilleure solution : un fichier de base (`docker-compose.yml`) définissant la configuration commune, complété par un fichier de surcharge (*override*) par environnement (`docker-compose.dev.yml`, `docker-compose.rec.yml`, etc.), combinés au déploiement via l'option `-f`²⁰. Cette approche par composition, versionnée sur une unique branche, rendrait explicites - dans un simple diff de fichiers plutôt qu'un git diff entre deux branches distantes - les différences réelles entre environnements, tout en garantissant qu'une correction commune ne soit écrite, et donc corrigée, qu'à un seul endroit.

²⁰`docker compose -f docker-compose.yml -f docker-compose.rec.yml up`

Cette remarque reste toutefois une lecture personnelle du schéma en place plutôt qu'un problème concrètement rencontré : je n'ai pas observé de cas réel où une telle synchronisation manuelle par *cherry-pick* ait été nécessaire pendant la durée de mon stage - l'anti-pattern est ici une déduction logique de la structure du dépôt, non un incident vécu.

III.1.5 Le processus de livraison et de montée de version

La livraison d'une nouvelle version de PIAWEB est une procédure méticuleuse qui demande rigueur et précision. La responsabilité de la création des livrables incombe à l'utilisateur système `piaweb` directement sur le serveur.

La procédure débute par la récupération des dernières modifications du code source depuis la branche de recette (`rec`) et de tous ses sous-modules. Le code dorsal est ensuite compilé via l'outil Maven (`mvn clean install`). Une fois les binaires générés, la construction des nouvelles images Docker (`backend`, `frontend`, et `flyway`) est lancée en désactivant le cache pour forcer une reconstruction totale.

Ces nouvelles images sont ensuite étiquetées (taggées) avec le numéro de version correspondant (par exemple `$CI_COMMIT_TAG`) et envoyées (poussées) vers le registre d'images privé d'Actimage (`piaweb-registry.actimage-ext.net`). L'accès à ce registre est protégé et nécessite une authentification préalable via la commande `docker login`.

Une fois les livrables créés et sécurisés sur le registre, la mise à jour effective de l'environnement (la montée de version) peut avoir lieu. Ce processus suit une chorégraphie immuable :

Arrêt des services

Les modules applicatifs en cours d'exécution sont stoppés proprement grâce à des scripts dédiés (`stop.sh`) présents dans l'arborescence de chaque composant.

Sauvegarde

Avant toute manipulation des données, une sauvegarde complète de la base de données PostgreSQL est réalisée (via un export `pg_dump` compressé en `.zst`), garantissant un point de restauration en cas de problème.

Configuration

Les fichiers de configuration d'environnement (`host.env`) des différents modules sont édités pour faire pointer la variable `VERSION` vers le nouvel identifiant de l'image Docker fraîchement produite.

Migration des données

Le script de migration Flyway (`flyway.sh migrate`) est exécuté. Il se charge d'appliquer séquentiellement les nouveaux scripts SQL nécessaires à la mise à jour de la structure ou des données de la base, tout en traçant son exécution.

Démarrage et contrôle

Les modules `backend` et `frontend` sont finalement relancés via leurs scripts `start.sh`. Le bon déroulement de l'opération est vérifié en inspectant les journaux de bord (logs) des conteneurs en temps réel. Il est impératif de lancer le module dorsal en premier puisqu'il peut fonctionner seul. Le module frontal, quant à lui, essaie automatiquement de se connecter au dorsal, pouvant entraîner un crash au démarrage si celui-ci n'est pas encore prêt à traiter des requêtes entrantes.

Si la recette est validée par le client, la version est promue en production. Les images testées sont simplement re-tagguées avec le préfixe `prod-` puis propulsées sur l'environnement de production, assurant ainsi qu'aucune modification de code n'a pu altérer l'application entre la phase de test et la mise en ligne finale.

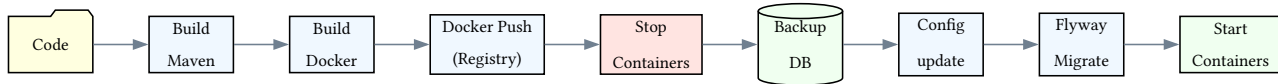


Fig. 14. – Schéma de flux illustrant les étapes de la livraison

III.2 Première livraison, première leçon

III.2.1 Le bogue

Étant l'un des développeurs les moins coûteux, j'ai été assigné à la correction d'un bogue d'affichage ordinaire pour lequel plusieurs de mes collègues avaient déjà imputé du temps. La plongée laborieuse dans le code source d'un projet dont la teneur m'échappait encore et dont le cadrage frontal ne m'était pas familier m'a contraint à optimiser mon débogage afin d'identifier la source du problème sans avoir à explorer l'entièreté de l'application. Quelques échanges de tickets avec le client plus tard et j'arrivais à reproduire le comportement anormal sur mon poste. Le problème venait d'un simple formulaire de recherche dont la pagination n'était pas rapportée à 1 lorsque l'utilisateur changeait les critères de recherche, permettant ainsi d'accéder à la troisième page pour une recherche ne remontant qu'une page de résultats par exemple.

Si l'implémentation du correctif ne nécessita que quelques minutes, la validation de la demande de fusion sur la branche de développement dev fut actée en une demi-heure. L'intervention aurait pu s'achever sur cette bonne note, mais l'équipe DevOps m'a confié la responsabilité de l'intégralité du cycle de livraison de cette version, incluant la montée sur les différents environnements et le déploiement final chez le client.

III.2.2 Appareillage sur lest

Le cycle de livraison d'une version du site PIAWEB passe par plusieurs phases différentes, évoluant lentement de l'environnement de développement vers l'environnement de production. Personne ne m'avait formellement transmis la procédure de livraison à ce moment-là - la documentation que je cite plus haut dans ce rapport a d'ailleurs été rédigée **après** les événements que je m'apprete à raconter, en partie grâce à eux. Sans cette référence, j'ai naturellement reproduit ce que je connaissais déjà : les mêmes commandes de mise à jour utilisées quotidiennement sur les environnements de développement et d'intégration, où l'application tourne en écoutant un volume monté contenant le code source, sans jamais rien figer dans l'image elle-même.

Ces commandes fonctionnent à merveille sur dev et int. Elles fonctionnent également très bien sur rec, pour une raison que j'ignorais alors : l'environnement de recette d'Actimage réutilise le même serveur, où le code se trouve donc toujours physiquement présent sur le disque, monté en volume comme sur les environnements précédents - même si en théorie, à ce stade du cycle de livraison, le code est censé être figé à l'intérieur de l'image elle-même. Les tests passaient, l'application tournait normalement sur int comme sur rec, rien ne laissait présager de problème. J'ai donc construit les images, poussé les artefacts sur le registre Harbor d'Actimage, et lancé la livraison chez le client.

Celle-ci a d'abord été ralentie par un problème indépendant de ma manipulation : un changement de configuration de proxy côté Actimage avait entre-temps rendu le registre inaccessible depuis le réseau du client. Une fois ce point réglé et les images enfin récupérées côté client, celles-ci ont tout simplement refusé de démarrer :

```
Error: Could not find or load main class Main
Caused by: java.lang.ClassNotFoundException: Main
```

J'ai reproduit l'erreur en local, sur mon propre ordinateur, après avoir pris soin de retirer les conteneurs de développement du projet pour ne pas fausser le test. Même résultat. En ouvrant un shell à l'intérieur du conteneur incriminé, la cause est devenue évidente : le répertoire /app, censé contenir le code compilé de l'application, était tout simplement absent. Mes images ne contenaient aucun code.

En suivant sans le savoir le protocole de build de dev et int jusqu'à rec puis prod, j'avais construit des images strictement vides de toute logique applicative - le code, sur le serveur d'intégration où j'opérais, restait monté en volume depuis le système de fichiers hôte, et n'était donc jamais copié à l'intérieur de l'image lors du

`docker compose build`. Rétrospectivement, le signal m'avait pourtant crevé les yeux sans que je le lise correctement : la taille des nouveaux artefacts sur le registre avait presque diminué de moitié par rapport à la version précédente - une image sans code pesant naturellement bien moins lourd qu'une image qui en contient.

Tout est rentré dans l'ordre une fois qu'on m'a expliqué la procédure exacte de bascule entre les deux paradigmes de construction, et une fois la documentation correspondante rédigée. Ce genre d'incident, sur un projet DevOps mature, ne pose généralement pas de difficulté majeure : il suffit de connaître un peu la stack pour en repérer rapidement la cause. Le problème, dans mon cas, tenait autant à l'absence ponctuelle de documentation qu'à ma propre formation insuffisante en DevOps, réseaux et en infrastructure.

Tout DevOps le sait, une bonne documentation n'est rien sans le savoir faire de celui qui la lit. Il y aura toujours un détail d'environnement qui changera ou une doc pas à jour qui forcera le DevOps à « mettre les mains dans le cambouis » et déboguer sur le serveur directement.

IV Autres projets en cours

IV.1 Automatisation de la qualification

Le projet de Romain vise à automatiser le processus de qualification d'un AO. Habituellement, une qualification prend au moins deux heures à un consultant : c'est une tâche répétitive et peu valorisante, dont le temps investi peut de surcroît être perdu si l'AO qualifié ne correspond finalement pas aux compétences des équipes d'Actimage. Qualifier un AO consiste concrètement à extraire un ensemble de données utiles d'un document PDF (souvent volumineux et peu structuré) pour les reporter dans une feuille de calcul standardisée, elle-même découpée en une dizaine de thématiques - contexte, procédure, durée et budget, critères de sélection, contacts - représentant au total plusieurs dizaines de champs à renseigner.

J'ai assisté à une présentation mi-technique mi-pratique de ce projet par Romain lui-même. Plutôt que de recourir à un grand modèle de langage généraliste, coûteux en puissance de calcul et disproportionné pour une tâche d'extraction aussi ciblée, son choix s'est porté sur Kimi [28], un modèle à poids ouverts publié par Moonshot AI, couplé à une architecture de RAG (*Retrieval-Augmented Generation*) reposant sur un découpage du document source en fragments (*chunking*) puis leur vectorisation (*embedding*). L'ensemble tourne en local, directement sur un Mac M1, ce qui ramène le temps de qualification d'un AO à une vingtaine de minutes. À ce rythme, qualifier un AO cesse d'être une perte de temps potentielle : la totalité des opportunités repérées en amont, lors de la phase de prospection commerciale, pourrait être systématiquement qualifiée, réduisant à néant le risque qu'Actimage laisse filer une occasion de répondre à un AO faute de temps disponible pour l'évaluer.

Le système, bien qu'abouti dans les grandes lignes, n'est pas encore utilisé en conditions réelles : Romain souhaite encore l'améliorer, notamment sur le traitement des AO à lots multiples, où les informations relatives à chaque lot sont entremêlées dans le document source et rendent le découpage en fragments (*chunking*) nettement plus délicat à fiabiliser. Un second projet, encore sans nom à ce jour, est par ailleurs en préparation : la récupération automatique des AO publiés sur les plateformes de marchés publics en ligne. Greffé à l'outil de qualification de Romain, il constituerait une chaîne de traitement complète, de la veille jusqu'à la qualification, qui ferait gagner un temps considérable à l'équipe business d'Actimage.

Ce projet illustre par ailleurs une conviction qui m'est chère : celle de la souveraineté numérique et de la maîtrise de ses propres données. L'un des objectifs affichés, au-delà du strict gain de temps interne, est de pouvoir à terme proposer une prestation similaire aux clients d'Actimage, avec l'argument tangible de l'utiliser déjà soi-même en interne. J'apprécie particulièrement cette philosophie du *Do It Yourself*, qui présente le double avantage d'être formatrice - construire son propre outil oblige à en comprendre chaque rouage - et de libérer d'une dépendance à un prestataire ou un fournisseur extérieur, particulièrement précieuse lorsque les données traitées (contenu d'AO, informations client) ne sont pas destinées à transiter par les serveurs d'un tiers.

Enfin, une remarque personnelle sur la documentation produite par Romain pour décrire précisément le contenu attendu de chaque cellule de la feuille de qualification : conçue à l'origine pour être ingérée par le système de RAG, je l'ai trouvée tout aussi efficace comme support pédagogique à destination d'un humain. N'ayant moi-même aucune connaissance préalable du processus de qualification d'AO, sa seule lecture m'en a donné une compréhension plus claire que n'importe quelle explication orale n'aurait pu le faire - un signe, je crois, qu'un document suffisamment précis pour guider une machine finit souvent par constituer, presque accidentellement, la meilleure documentation possible pour un nouvel arrivant.

V Conclusion

V.1 À propos de PHP et Symfony : une cacophonie harmonisée

V.1.1 Une histoire de typage progressif

PHP n'était à l'origine pas fortement typé. Des types ont progressivement été ajoutés aux frontières des contrats - paramètres et valeurs de retour des fonctions, propriétés de classe - mais ce typage ne fait, par défaut, que convertir (*cast*) implicitement les valeurs fournies plutôt que de rejeter les mauvais types. Il faut déclarer `declare(strict_types=1)` ; en tête de chaque fichier pour que l'interpréteur fasse effectivement échouer l'exécution lorsqu'un type incorrect est transmis.

Aujourd'hui, PHPStan embarque un système de types nettement plus puissant et expressif, permettant notamment l'inférence de types - à la manière de langages modernes comme Rust, Java avec `var` [5], ou C++ avec `auto` [6]. Cette fonctionnalité réduit à la fois le volume de code à écrire et la quantité d'information explicitement affichée à l'écran. L'avènement des serveurs de langage a largement résorbé ce problème dans l'éditeur ; il persiste néanmoins dès que le code est lu hors d'un environnement supportant le LSP - imprimé, affiché en ligne, ou dans tout autre contexte dépourvu d'inférence assistée.

V.1.2 Un parallèle avec l'écosystème JavaScript

JavaScript, autre langage initialement faiblement typé, a suivi un arc de rédemption comparable, mais plus rapide et désormais achevé avec l'arrivée de TypeScript. Ce sur-ensemble du langage apporte, à l'image de PHPStan, une rigueur de typage sans altérer le comportement du code à l'exécution - au prix d'une étape de transpilation vers JavaScript, un coût toutefois négligeable dans un écosystème déjà rompu à des étapes de traitement comparables (minification, *bundling*).

Cette étape de transpilation a d'ailleurs commencé à s'effacer récemment, avec l'apparition d'environnements d'exécution capables d'interpréter directement la syntaxe TypeScript sans pour autant en vérifier les types - un fonctionnement qui n'est pas sans rappeler celui de l'interpréteur PHP, qui ignore purement et simplement les types PHPStan exprimés en commentaires PHPDoc. Peut-être verra-t-on un jour un interpréteur PHP capable de comprendre nativement l'ensemble des types complexes de PHPStan ; Facebook avait ouvert cette voie dès 2014 avec son propre langage, Hack [29].

V.1.3 Symfony face à Spring Boot

Structurellement, Symfony ressemble beaucoup à Spring Boot : ORM, contrôleurs, dépôts, entités et services y jouent des rôles quasiment identiques. Les deux frameworks s'appuient sur des métadonnées déclaratives posées directement sur le code - les annotations pour Spring Boot, les attributs pour Symfony -, mais leur traitement diffère fondamentalement. PHP n'étant pas un langage compilé, ces attributs sont évalués et leur comportement appliqué à l'exécution, via la réflexion et l'introspection. Spring Boot recourt lui aussi à la réflexion, mais un usage extensif à des fins purement applicatives y est généralement déconseillé ; Symfony s'appuie sur ce mécanisme de façon plus systématique encore, faute d'un traitement à la compilation comparable à celui que permettrait, en Java, un outil comme Project Lombok.

Venant de Java, la prise en main de Symfony s'est révélée aisée et agréable. L'un des atouts de PHP tient à ses tableaux associatifs, équivalents des objets JavaScript, capables de transporter une donnée de forme quelconque - un atout particulièrement appréciable pour la configuration de composants, où l'ensemble des options peut s'écrire de façon concise et lisible sous la forme `['class' => FooBar::class, 'required' => false, 'name' => $name]`, quand l'équivalent Java prendrait la forme d'un enchaînement `.setClass(Foobar.class).setRequired(true).setName(name)`. La configuration au format YAML relève de la même philosophie : plus lisible que le XML, plus expressive que les fichiers `application.properties` de Java.

Cette flexibilité se prolonge dans l'extensibilité de l'écosystème. Là où un projet Spring Boot se limite généralement à quelques archives jar sur lesquelles construire l'application, Symfony ouvre l'accès à un univers

de paquets comparable à celui de l'écosystème JavaScript : à chaque fonctionnalité souhaitée correspond, le plus souvent, un paquet existant - parfois maintenu par Symfony lui-même. Cette richesse a toutefois un revers : elle expose davantage les applications aux attaques de la chaîne d'approvisionnement (*supply-chain attacks*).

Sur la question du typage, PHP conserve malgré tout un atout que Java ne possède pas : les traits. Un trait constitue une solution moins propre qu'un héritage ou qu'une composition à part entière, mais dans le cadre d'un projet mené en agilité, où l'ensemble des besoins n'est pas connu dès le premier jour, il permet de ne pas imposer prématurément des structures logicielles qui freineraient le développement ultérieur. Une fois un projet PHP arrivé à maturité, rien n'empêche de migrer vers de l'héritage là où cela devient pertinent ; mais durant les phases de développement rapide, cette rigidité anticipée constituerait un frein plutôt qu'un gain. L'héritage va en ce sens de pair avec l'encapsulation : une fonctionnalité achevée se fige dans un module, un paquet ou une classe Java fermée, dont le code n'aura plus vocation à évoluer, sauf changement ultérieur des spécifications qui la concernent directement.

Le typage de PHPStan se révèle par ailleurs plus expressif que celui de Java sur un point précis : une référence Java peut toujours être nulle, sans qu'il existe de qualification de type sensible au flux d'exécution - comme le permettent TypeScript ou PHPStan - où une assertion de non-nullité change effectivement le type inféré, du point de vue du serveur de langage comme du compilateur. Cette distinction se joue en revanche bel et bien à l'exécution côté JVM, qui optimise différemment la mémoire allouée à une référence vide.

V.1.4 Le défi de la performance de l'outillage

L'ensemble de ces outils - PHPStan en tête - permet de développer des systèmes PHP plus complexes sans que la fiabilité n'en pâtisse. Se pose alors la question de leur passage à l'échelle sur de gros projets : les outils de développement (serveurs de langage, formateurs, *linters*, analyseurs statiques) deviennent rapidement trop lents - les diagnostics de PHPactor accusent souvent un retard perceptible dans l'éditeur, notamment parce que le serveur s'appuie lui-même sur PHPStan pour une partie de ses vérifications.

L'écosystème JavaScript/TypeScript a déjà trouvé une réponse à ce problème : remplacer ces outils par des réécritures modernes dans des langages plus performants, tels que Rust (Biome [26], oxc [30]) ou Go (TypeScript 7[31]). J'espère voir cette tendance se propager à l'écosystème PHP. Le projet PHPantom, déjà mentionné plus haut dans ce rapport, en constitue un exemple prometteur : un serveur de langage PHP développé en Rust, nettement plus rapide au démarrage et à la réponse que PHPactor. Une limite demeure cependant : l'analyse de type performante de PHPStan repose fondamentalement sur la réflexion et l'introspection du code PHP lui-même, une capacité qu'il paraît difficile de réimplémenter fidèlement dans un autre langage, à moins de greffer une extension directement à l'interpréteur PHP, le Zend Engine [32].

V.2 Enrichissement personnel

TODO: parler de l'envie de bosser à la dinum, souveraineté numérique, FabPot

Bibliographie

- [1] Institut National Polytechnique de Toulouse (Toulouse INP), « Identité visuelle et outils de communication ». [En ligne]. Disponible sur: <https://www.inp-toulouse.fr/>
- [2] Actimage, « Identité visuelle et outils de communication ». [En ligne]. Disponible sur: <https://www.actimage.com/>
- [3] Contributeurs de Wikipédia, « Office national des combattants et des victimes de guerre ». [En ligne]. Disponible sur: https://fr.wikipedia.org/wiki/Office_national_des_combattants_et_des_victimes_de_guerre
- [4] Linus Torvalds, « Manpage Git - Section git-commit ». [En ligne]. Disponible sur: <https://git-scm.com/docs/git-commit#Documentation/git-commit.txt---no-verify>
- [5] Oracle, « Local Variable Type Inference ». [En ligne]. Disponible sur: <https://docs.oracle.com/en/java/javase/21/language/local-variable-type-inference.html>
- [6] cppreference.com contributors, « Placeholder type specifiers ». [En ligne]. Disponible sur: <https://cppreference.com/cpp/language/auto>

Références logicielles

- [7] Figma, « Figma ». [En ligne]. Disponible sur: <http://figma.com/>
- [8] Microsoft, « Windows Subsystem for Linux ». [En ligne]. Disponible sur: <https://wsl.dev/>
- [9] JetBrains, « PhpStorm ». [En ligne]. Disponible sur: <https://www.jetbrains.com/phpstorm>
- [10] JetBrains, « IdeaVim ». [En ligne]. Disponible sur: <https://lp.jetbrains.com/ideavim>
- [11] « Neovim ». [En ligne]. Disponible sur: <https://neovim.io/>
- [12] Microsoft, « LSP ». [En ligne]. Disponible sur: <https://microsoft.github.io/language-server-protocol>
- [13] Daniel Leech, « Phpactor ». [En ligne]. Disponible sur: <https://phpactor.readthedocs.io/>
- [14] Ben Mewburn, « Intelephense ». [En ligne]. Disponible sur: <https://intelephense.com/>
- [15] Anders Jenbo, « Phpantom ». [En ligne]. Disponible sur: https://phpantom-dev.github.io/phpantom_lsp
- [16] Rust Team, « Rust ». [En ligne]. Disponible sur: <https://rust-lang.org/>
- [17] Fabien Potencier, « Symfony Language Tools ». [En ligne]. Disponible sur: <https://github.com/symfony/language-tools>
- [18] Mikhail Gunin, « Twiggy ». [En ligne]. Disponible sur: <https://github.com/moetelo/twiggy>
- [19] « Tree-sitter ». [En ligne]. Disponible sur: <https://tree-sitter.github.io/>
- [20] Andy Massimino, « Vim Matchup ». [En ligne]. Disponible sur: <https://github.com/andymass/vim-matchup>
- [21] « tmux ». [En ligne]. Disponible sur: <https://github.com/tmux/tmux>
- [22] Jesse Duffield, « Lazygit ». [En ligne]. Disponible sur: <https://github.com/jesseduffield/lazygit>
- [23] Jesse Duffield, « Lazydocker ». [En ligne]. Disponible sur: <https://github.com/jesseduffield/lazydocker>
- [24] Docker Inc., « Docker ». [En ligne]. Disponible sur: <https://www.docker.com/>
- [25] Microsoft, « Microsoft Access ». [En ligne]. Disponible sur: <https://www.microsoft.com/fr-fr/microsoft-365/access>
- [26] « Biome ». [En ligne]. Disponible sur: <https://biomejs.dev/>
- [27] Microsoft, « Playwright ». [En ligne]. Disponible sur: <https://playwright.dev/>
- [28] Moonshot AI, « Kimi ». [En ligne]. Disponible sur: <https://www.kimi.ai/>
- [29] Facebook, « Hack ». [En ligne]. Disponible sur: <https://hacklang.org/>
- [30] « Oxc - A collection of of high-performance JavaScript tools written in Rust ». [En ligne]. Disponible sur: <https://oxc.rs/>
- [31] Microsoft, « Announcing TypeScript 7.0 ». [En ligne]. Disponible sur: <https://devblogs.microsoft.com/typescript/announcing-typescript-7-0>
- [32] « Zend Engine ». [En ligne]. Disponible sur: https://www.phpinternalsbook.com/php7/zend_engine.html